

LANTERN: Diagnosing Web Intrusion Drift under Evolving Attackers

Anonymous Authors

Abstract—Web attackers and tooling evolve continuously, degrading intrusion detection models as their internal representations fail to adapt. Existing drift detectors are brittle in production, either flooding operators with false alarms or missing genuine, structural shifts. We introduce LANTERN, an interpretable, dual-channel drift monitor that diagnoses when and why models degrade using two complementary indicators: *Latent Mahalanobis Typicality* (LMT), which captures deformation in the representation space, and *Predictive Entropy* (PE), which quantifies decision level confusion. We evaluate LANTERN on 32.9 million continuous HTTP(S) requests collected over 7 months by a large-scale distributed honeyfarm and find that prominent state-of-the-art detectors are operationally unviable, generating thousands of false positives. In contrast, LANTERN’s signals align tightly with true degradation: LMT attains a correlation of $r = 0.938$ with F1 score, and our adaptive deployment maintains stable accuracy over time with only 1.93% triggering rate, ending the period with an F1 score more than 10% higher than a static model. Beyond statistical metrics, LANTERN uncovers the operational roots of drift by identifying acute structural shifts such as a large centralized distribution event and tracing the evolution of drift associated exploit behaviours through longitudinal analysis. These results position LANTERN as a production ready and interpretable monitor for web intrusion detection under evolving threats.

Index Terms—Intrusion Detection, Web Security, Drift, Honeybots, Latent Representations, Predictive Entropy

1. Introduction

The modern web ecosystem is in constant flux, shaped not only by legitimate changes in service usage but also by the continuous evolution of malicious activity. Recent measurements show that tens of thousands of websites are compromised daily and hundreds of thousands of new malware samples appear worldwide [1]. This relentless pace of evolution reshapes the network traffic observed by intrusion detection systems (IDS) and introduces behavioural changes that accumulate over time, leading to model degradation [2]. These resulting changes are broadly defined as *drift*. Unlike conventional notions such as distributional drift [3]–[6] or concept drift [2], [7]–[9], which often focus on shifts in features or labels, the drift we observe arises from evolving attacker behaviour within web traffic.

Existing drift detection methods, whether statistical [5], [10], distance based [8] or uncertainty driven [11], perform well on curated benchmarks where data is clean and attacks are isolated [12]–[16]. In real web environments the assumptions behind these detectors break down,

leading to unstable signals and degraded reliability. Reliable and interpretable indicators of model degradation are therefore essential for maintaining operational resilience in evolving web contexts.

HTTP(S) traffic amplifies these difficulties. Requests mix multiple campaigns [17], partial obfuscation [18], [19] and rapidly shifting payload and URI structures [20], producing streams where behavioural changes are subtle and noisy. This complexity makes drift difficult to isolate and requires indicators that remain stable while capturing meaningful patterns in attacker activity.

We take the view that diagnosing drift requires indicators that reflect behavioural change rather than abstract shifts in features or labels. New payload families, staging patterns and structural variations reshape the latent space in ways that conventional detectors do not capture. Tracking these behavioural processes provides a clearer basis for recognising when an IDS is beginning to lose alignment with evolving attacker activity.

We introduce LANTERN (acronym for LAtE nT Typicality and EntRopy moNitor), an interpretable multi channel monitor that identifies when an intrusion detection model begins to lose alignment with evolving attacker activity and characterises the nature of this degradation through complementary signals. Built on a contrastive autoencoder based detection pipeline, the predictive entropy channel captures decision level confusion by measuring uncertainty in model outputs, while the latent typicality channel measures structural deformation in the representation space associated with different activity patterns. Together these indicators provide a stable and explainable characterisation of drift across decision and representation layers, supporting informed analysis of evolving behaviour and guiding adaptive response when needed. Taken together, these components form the basis of our study. Our contributions are as follows:

- We present a longitudinal analysis of malicious drift in real-world HTTP(S) traffic, drawing on seven months of data from a large-scale, globally distributed honeypot deployment.
- We provide a systematic assessment of existing drift detection paradigms under non-stationary and adversarial conditions, revealing their instability and operational limitations in real web streams.
- We introduce LANTERN, a multi channel and interpretable framework that combines predictive entropy and latent typicality to capture model confusion in the IDS layer and representation deformation in the latent space, providing a coherent characterisation of degradation across both levels.
- We validate the practical relevance of these indicators through case studies that reveal coordinated distribution waves, propagation bursts and ex-

exploitation chains, showing that they capture meaningful behavioural patterns in real web traffic.

Our code is available at <https://anonymous.4open.science/r/LANTERN-4DC3/>.

2. Threat Model and Research Questions

2.1. Threat Model

We consider a large-scale, distributed web honeypot deployment that continuously collects HTTP(S) traffic. On top of this data plane an intrusion detection model maps each request into a latent representation and produces a class prediction. A monitoring component evaluates how the model and its latent space evolve over time. Our focus is on understanding how attacker behaviour shapes this evolution and how drift emerges in practice.

Defender. The defender operates the honeypot infrastructure and controls feature extraction, model training and monitoring. They can log all traffic and maintain historical statistics, but cannot depend on manual inspection of alerts or frequent human supervision. Their goal is to keep detection performance stable as behaviour evolves, relying on monitoring signals that are reliable and interpretable rather than dense or unstable.

Adversary. The adversary consists of remote web attackers who generate evolving malicious HTTP(S) traffic towards our testbed. They influence the stream by modifying payload syntax through encoding or obfuscation, introducing new exploit variants or changing target semantics, and altering traffic characteristics such as timing and volume. These behavioural changes reshape the observable HTTP(S) traffic and gradually shift the distribution encountered by the model. The adversary cannot modify the honeypot infrastructure or internal model parameters; their impact arises solely from the evolving structure of the requests they generate, which induces *malicious drift* that moves the model away from its training regime.

Drift Surface. These evolving behaviours give rise to two interconnected forms of drift. (i) *Representation drift*: the latent structure of HTTP(S) traffic changes as new payload families, staging patterns and request templates appear, shifting the shape, spread and position of latent clusters. (ii) *Decision drift*: the classifier becomes increasingly uncertain or misaligned, producing higher confusion even when surface level changes appear subtle. Benign evolution in web usage forms a noisy background, but our primary interest is in how malicious drift accumulates and pushes the model away from its training regime.

2.2. Research Questions

Before introducing our research questions we first outline the limitations of existing approaches. Current drift detectors were not designed for adversarial, noisy and long-term web environments. Distance based approaches rely on fixed thresholds in latent space, reconstruction error metrics often convert minor statistical fluctuations into continuous alarms and generic uncertainty measures provide limited operational meaning. In practice these methods either track drift poorly or generate unsustainable alert volumes. This motivates a set of questions about

whether drift signals can reflect model health reliably in such settings. We therefore structure our study around the following research questions.

RQ1: Indicator Fidelity. *Do existing and newly proposed drift indicators faithfully track actual model degradation over long time horizons in real web traffic?*

Prior work largely evaluates drift detectors on synthetic or short-lived datasets, measuring statistical detection accuracy rather than alignment with end-to-end model performance over time. We instead ask whether drift indicators can act as *model health sensors*: when the model deteriorates, do their scores move in a stable and monotonic fashion, and when the model is stable, do they remain quiet? This highlights a gap in the literature, where the alignment between drift scores and actual IDS performance under real web traffic remains largely unexplored.

RQ2: Operational Reliability. *Can drift indicators be tuned to avoid overwhelming operators with false alarms while still catching genuine degradation events?*

Most existing detectors are evaluated with loose thresholding and the resulting alert workload is rarely examined. In noisy honeypot environments a detector that raises large volumes of alarms is unusable even if many are statistically correct. We therefore ask whether practical thresholding schemes can produce a usable balance between missed and raised drift events, and how different paradigms perform under this operational view. This question stresses a missing dimension in the state of the art: drift detection that is not only accurate in theory, but also manageable in practice.

RQ3: Drift-Aware Adaptation Effectiveness. *Does using drift signals to trigger adaptive retraining lead to more stable long-term intrusion detection performance than static or naively scheduled retraining?*

While many studies introduce drift detectors, far fewer examine whether acting on these signals improves the robustness of deployed models. Figure 1 motivates this question by showing that a static model undergoes both transient performance collapses and sustained long-term degradation in a real honeypot deployment. However, it remains unclear whether drift-driven adaptation can prevent such drops without overfitting to short-term fluctuations or causing regressions. We therefore ask whether a drift-aware strategy can maintain higher and more stable accuracy and F1 over months of evolving web traffic and how it compares to state-of-practice approaches.

RQ4: Generalisation Under Heterogeneous Conditions. *Do drift indicators and adaptation strategies remain reliable when traffic sources change, and do they retain meaningful behaviour across different attack classes?*

Many evaluations of drift methods rely on homogeneous or synthetic streams, which fail to capture the variability present in real web traffic. Our deployment integrates traffic from multiple sensors and spans diverse attack families over months. We therefore ask whether drift indicators preserve their alignment with model performance when new traffic streams are introduced, where heterogeneity naturally triggers drift alerts, and whether adaptive updates can successfully incorporate these new sources into the model. We also examine whether structural signals remain interpretable across different classes

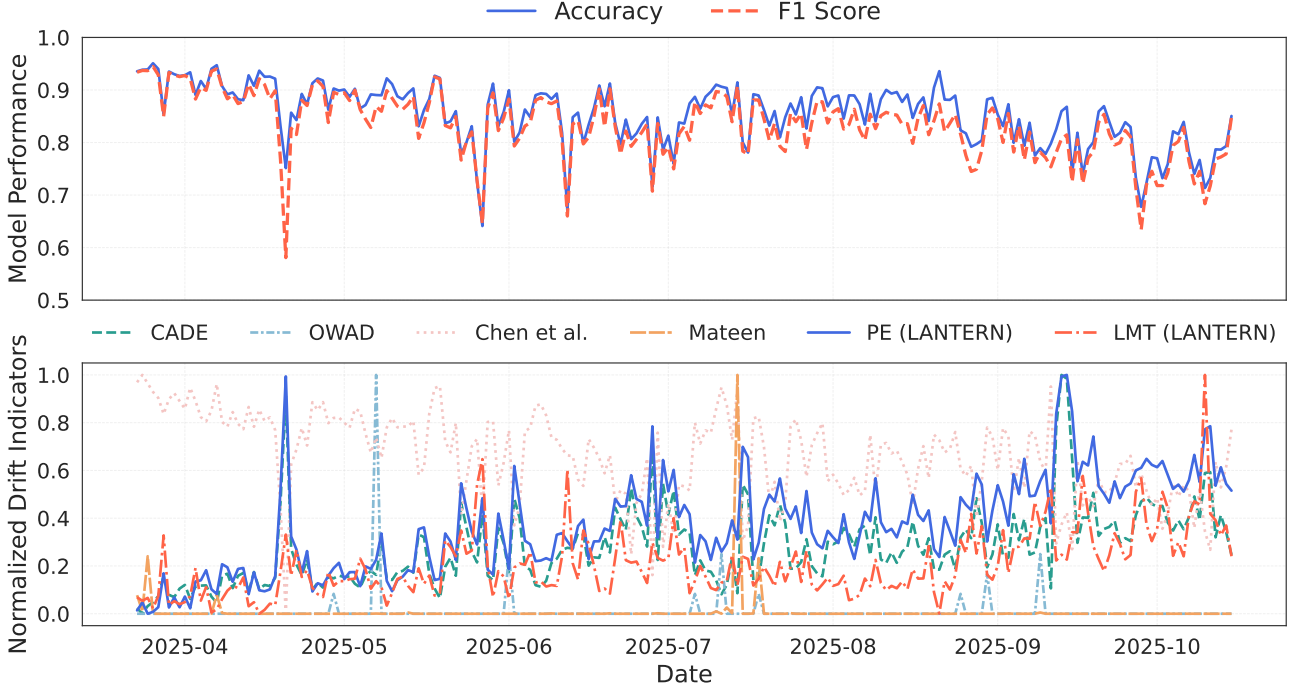


Figure 1: Model performance (top) and normalized drift indicators (bottom) over time. The upper subplot presents F1 Score and Accuracy, while the lower subplot compares normalized drift indicators across methods. The drift is evident from the overall trends of both model performance degradation and indicator dynamics.

and can extend to newly defined behavioural categories as the threat landscape evolves. This probes the extendability of our framework as the real-world traffic becomes increasingly heterogeneous.

Together, these research questions shift the focus from purely statistical drift detection towards *operationally meaningful* drift diagnosis in web intrusion detection: indicators must faithfully reflect model health, generate manageable alerts, enable effective adaptation, and remain robust across heterogeneous, evolving traffic.

3. LANTERN: Designing a Dual-Channel Drift Monitor

This section presents the design of LANTERN, an interpretable and adaptive drift-monitoring framework tailored for adversarial web environments. We formalise the operational setting, introduce the two complementary indicators that form the core of the system, and describe the adaptive logic that enables reliable, low-noise intervention.

3.1. Problem Formulation and Notation

We begin by formalising the drift-diagnosis setting for non-stationary traffic streams collected by distributed web honeypots. We first define the operational testing paradigm, characterize the forms of malicious behaviour shifts considered, and introduce the mathematical notations used throughout the design.

3.1.1. Detection Paradigm: Reference vs. Test Stream. We formulate the drift diagnosis problem as a continuous

consistency check between a stable *Reference Buffer* and the incoming *Livestream*.

The buffer represents the baseline distribution on which the model operates, and the livestream supplies the current block for evaluation.

- **Reference Buffer ($\mathcal{W}_{\text{ref}}$):** We define the reference set as the most recent N_{ref} blocks used for model training. In our implementation, $N_{\text{ref}} = 192$ consecutive blocks (8 days), which form the operational baseline. The buffer is reconstructed after each confirmed drift event.
- **Test Block ($\mathcal{W}_t$):** We define the test set as the single current block of traffic arriving from the livestream at time t .

The objective is to determine if the behavioural properties of $\mathcal{W}_t$ deviate significantly from the baseline defined by $\mathcal{W}_{\text{ref}}$. This comparison is projected into two complementary indicator spaces: *Latent Structure*, which quantifies deformation in representation space, and *Predictive Belief*, which quantifies confusion in classifier outputs.

3.1.2. Behavioural Drift in Adversarial Web Traffic. Attackers strategically evolve to evade detection, producing two characteristic forms of drift captured by our indicators:

- **Structural Drift (Payload Mutation $\rightarrow$ Latent Deformation).** Changes in payload syntax, obfuscation strategies, or toolchains produce latent vectors z that fall into low-density regions, far from the learned class structure. In LANTERN, this manifests as elevated *Latent Mahalanobis Typicality* (LMT).

- **Belief Drift (Mimicry $\rightarrow$ Decision Ambiguity).** Mimicry of legitimate traffic patterns increases classifier uncertainty, flattening output distributions. In our framework, this manifests as increased *Predictive Entropy* (PE).

3.1.3. Notations. Table 1 summarises the notation used to formalize the LANTERN pipeline.

TABLE 1: Notation used in the design of LANTERN.

Notation	Description
N_{ref}	Fixed Reference Window size.
$\mathcal{W}_{\text{ref}}$	Reference Buffer for training and baseline stats.
$\mathcal{W}_t$	Current block of incoming traffic.
x	Input feature encoded by TF-IDF
z	Latent representation from the Contrastive Autoencoder
$f(\cdot), g(\cdot)$	Contrastive encoder $f: x \rightarrow z$ and classifier $g: z \rightarrow \mathbf{p}$.
Z	Set of latent vectors $\{z_1, z_2, \dots, z_N\}$ in a block.
P	Set of probability vectors $P = \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_N\}$ in a block.
μ_c, Σ_c	Class Structure (Centroid μ_c and Covariance Σ_c).
$T(z)$	Squared Mahalanobis Distance (LMT score).
$H(z)$	Sample Predictive Entropy (PE score).
$S_{\text{LMT}}, S_{\text{PE}}$	Block Scores (Aggregated LMT and PE scores).
$\tau_{\text{LMT}}, \tau_{\text{PE}}$	Adaptive Decision Thresholds derived from $\mathcal{W}_{\text{ref}}$.
D_t	Binary indicator of confirmed drift at time t .

3.2. Dual-Channel Indicators for Drift Diagnosis

LANTERN employs two complementary indicators: one structural, one probabilistic, to diagnose when and why an intrusion detection model degrades.

3.2.1. LMT: Structural Drift Signal. LMT measures how well samples align with the structural shape of each class in the latent space. Unlike methods that rely on global Euclidean distance [8], LMT models each class by its centre and covariance structure. This allows the indicator to distinguish genuine structural drift (deformation) from natural, class-specific variation. Figure 2(a) illustrates this behaviour by showing how typicality increases as samples move away from the latent region of a class.

Latent shape estimation. We encode the training window and model each class c by a centroid μ_c and a covariance matrix Σ_c estimated using Ledoit–Wolf regularisation:

$$\mu_c = \frac{1}{|Z_c|} \sum_{z \in Z_c} z, \quad \Sigma_c = \text{LW}(Z_c) + \varepsilon I.$$

These parameters define an ellipsoidal region for each class, capturing its specific spread and orientation in the latent space.

Typicality. Given a latent sample z associated with class c , we measure its structural deviation using the squared Mahalanobis distance:

$$T(z) = (z - \mu_c)^T \Sigma_c^{-1} (z - \mu_c).$$

High values indicate an increasing mismatch with the learned class structure. As illustrated in Figure 2(a), samples deviating from the ellipsoid exhibit high typicality, while samples within the distribution remain low. This makes the indicator sensitive to structural departures.

Block level score. For each operational block, we compute the weighted mean typicality

$$S_{\text{LMT}} = \frac{1}{N} \sum_{i=1}^N T(z_i),$$

and estimate the proportion of samples with unusually large typicality values. These aggregated measures capture both broad latent deformation and concentrated structural anomalies within the block.

3.2.2. PE: Belief Drift Signal. PE measures the uncertainty of the classifier and provides a belief signal for drift. For each latent representation z , the classifier outputs a probability vector p , and the PE is defined as:

$$H(z) = - \sum_{c=1}^C p_c \log p_c.$$

Low PE indicates confident predictions, while high PE corresponds to model confusion. Figure 2(b) illustrates these regions via voting probability patterns.

Block level score. We compute PE for every sample in a block and use the mean value

$$S_{\text{PE}} = \frac{1}{N} \sum_{i=1}^N H(z_i)$$

as the block-level belief signal. Rising block PE often indicates emerging behaviour that shifts decision boundaries or introduces ambiguous features.

3.2.3. Reference Construction. Both indicators rely on baseline statistics computed from the reference window $\mathcal{W}_{\text{ref}}$. To support a layered detection strategy, balancing early awareness with precise intervention, we define distinct thresholds for monitoring and decision making.

LMT reference. Given the potentially heavy tailed nature of structural deviations, we summarise the distribution of S_{LMT} using the median and the median absolute deviation (MAD):

$$\begin{aligned} \tilde{m}_{\text{LMT}} &= \text{median}(S_{\text{LMT}}), \\ \text{MAD}_{\text{LMT}} &= 1.4826 \cdot \text{median}(|S_{\text{LMT}} - \tilde{m}_{\text{LMT}}|). \end{aligned}$$

Based on empirical tuning, we set both the monitoring and decision baselines at the robust 3σ :

$$\tau_{\text{LMT}} = \tilde{m}_{\text{LMT}} + 3.0 \cdot \text{MAD}_{\text{LMT}}.$$

PE reference. We characterise the distribution of block-level PE scores S_{PE} in the reference window by the mean and standard deviation:

$$\mu_{\text{PE}} = \text{mean}(S_{\text{PE}}), \quad \sigma_{\text{PE}} = \text{std}(S_{\text{PE}}).$$

Then we define a sensitive monitoring threshold at 2σ and a strict decision threshold at 3σ :

$$\tau_{\text{PE}}^{\text{mon}} = \mu_{\text{PE}} + 2.0 \cdot \sigma_{\text{PE}}, \quad \tau_{\text{PE}}^{\text{dec}} = \mu_{\text{PE}} + 3.0 \cdot \sigma_{\text{PE}}.$$

3.3. Adaptive Intervention Framework

The reliability analysis in Section 5.3 gives the insights that single indicator detection is limited for the robust adaptation for a balanced reliability and sensitivity. To address this, our framework employs a dual layer logic to manage the trade-off between operational awareness and automated intervention.

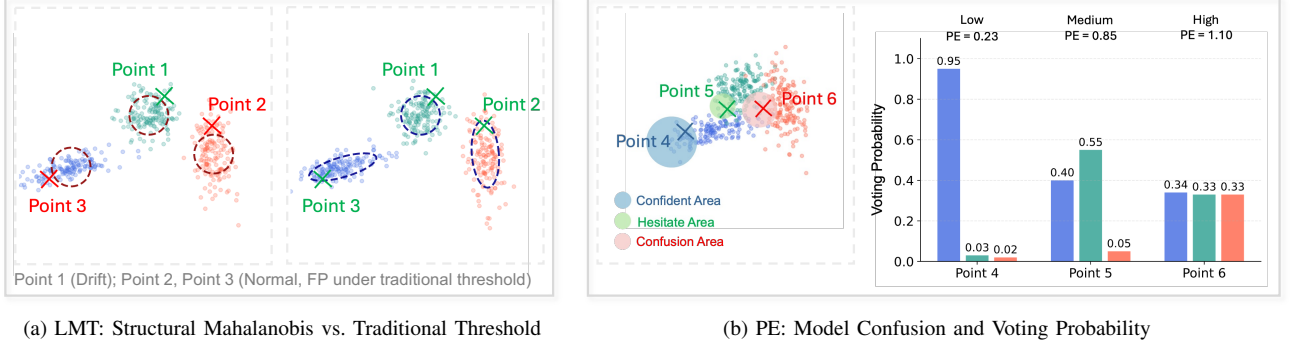


Figure 2: Conceptual view of the dual-channel indicators. (a) LMT captures deformation relative to class-specific latent geometry. (b) PE captures uncertainty and confusion in classifier outputs.

3.3.1. Monitoring Layer: Early Warning. The monitoring layer tracks potential instability without immediately triggering costly retraining. It flags a block as unstable if the PE exceeds the sensitive threshold τ_{PE}^{mon} (2.0σ) or if LMT exceeds τ_{LMT} (3.0σ). As shown in Table 7, this captures broader belief fluctuations (2.38%) and structural anomalies (23.18%) for operator visibility.

3.3.2. Decision Layer: High-Confidence Drift Confirmation. To prevent false alarms from triggering automated updates, the decision layer employs a stricter *Fusion Rule*. A drift event D_t is confirmed only when there is high confidence agreement between indicators or sustained structural failure. For an incoming block t , drift is confirmed if:

$$D_t = \left(S_{PE,t} > \tau_{PE}^{dec} \wedge S_{LMT,t} > \tau_{LMT} \right) \vee \left(\sum_{i=t-9}^t \mathbf{1}(S_{LMT,i} > \tau_{LMT}) \geq 8 \right).$$

When $D_t = 1$, the system reconstructs the reference buffer using the most recent N_{ref} blocks, retrain the model, and recomputes reference statistics. The first term captures joint high confidence drift while the second term captures persistent structural change. This ensures that retraining is only triggered by severe, high-confidence degradation, maintaining system stability. The complete procedure is detailed in Algorithm 1.

3.4. System Overview

Figure 3 summarises the architecture of our drift aware intrusion detection framework. The system operates in three layers that work together to monitor behavioural changes in honeypot traffic and support continuous adaptation.

Data Layer. The system collects HTTP(S) sessions from distributed honeypot sensors. The livestream supplies the current blocks used for detection, while the reference buffer maintains the recent baseline data on which the model operates. This buffer is updated only when confirmed drift occurs, after which PE and LMT references are recomputed for the next adaptation cycle.

Model Layer. This layer contains the core intrusion detection model. Each incoming block is transformed using a TF-IDF encoder and passed through a contrastive

Algorithm 1 LANTERN Adaptive Framework

Require: Stream blocks $\mathcal{B}$, reference buffer $\mathcal{W}_{ref}$

- 1: Train model M on $\mathcal{W}_{ref}$
- 2: Compute references and thresholds
- 3: Initialize structural-history buffer $\mathcal{Q}_{LMT}$
- 4: **for** each block B_t **do**
- 5: Encode $Z \leftarrow f(B_t)$ and predict $P \leftarrow g(Z)$
- 6: Compute $S_{PE,t}, S_{LMT,t}$
- 7: Append $\mathbf{1}(S_{LMT,t} > \tau_{LMT})$ to $\mathcal{Q}_{LMT}$
- 8: $\delta_{joint} \leftarrow (S_{PE,t} > \tau_{PE}^{dec})$
- 9: $\quad \wedge (S_{LMT,t} > \tau_{LMT})$
- 10: $\delta_{persist} \leftarrow \sum \mathcal{Q}_{LMT} \geq 8$
- 11: **if** δ_{joint} or $\delta_{persist}$ **then**
- 12: Rebuild $\mathcal{W}_{ref}$ and retrain M
- 13: Recompute thresholds and clear $\mathcal{Q}_{LMT}$
- 14: **end if**
- 15: **end for**

autoencoder to generate latent representations from the input features. These latent vectors are then processed by a classifier to give a detection decision.

Drift Detection Layer. This layer monitors changes using two complementary indicators that capture different aspects of drift. LMT measures structural deviation in latent space, and PE reflects belief changes in classifier outputs. Both signals are evaluated at the block level to assess consistency with the operational baseline. When the indicators provide strong or persistent evidence of drift, the system triggers the retrain mechanism and updates the reference buffer with data from the new model, allowing the framework to remain aligned with evolving traffic.

4. Testbed

Our testbed combines a 7-month longitudinal deployment of globally distributed web honeypots with an auxiliary heterogeneous dataset. Together, these sources provide the continuous, adversarial, and naturally evolving traffic necessary to study drift behaviour and assess adaptive detection under realistic conditions.

4.1. Primary Global Honeypot Deployment

The primary evaluation dataset is drawn from a long-running HTTP(S) honeypot deployment operating across

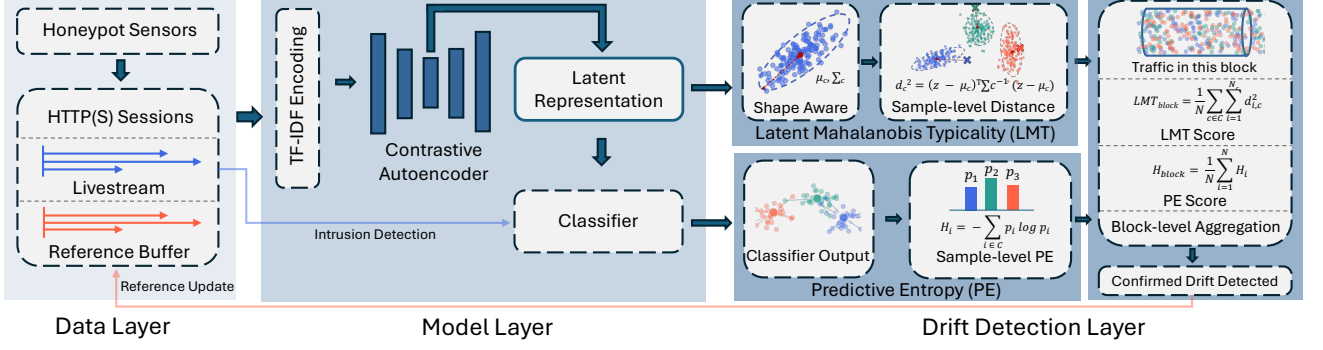


Figure 3: LANTERN system architecture. The data layer streams honeypot traffic and maintains the reference buffer. The model layer encodes and classifies traffic using a contrastive autoencoder pipeline. The drift-detection layer evaluates LMT and PE to guide adaptive retraining.

TABLE 2: Summary Statistics of the Primary Honeyplot Dataset

Stats	Value
Total Requests	32,894,941
Unique Client IP addresses	295,094
Unique Host IP addresses	271
Unique Countries (client)	194
Unique Countries (host)	25
Date Range	2025-03-15 – 2025-10-15

25 countries (Figure 4). This infrastructure produces a continuous and inherently non-stationary traffic stream, enabling a detailed measurement of drift dynamics and the stability of adaptive detection in the wild.

The deployment spans seven uninterrupted months and includes sensors hosted across diverse autonomous systems. Table 2 summarises dataset statistics. Honeyplot hosts emulate realistic web-service environments, including standard Linux server configurations, popular web stacks (e.g., Apache, WordPress), and behaviourally consistent HTTP(S) response patterns. These characteristics increase interaction fidelity and attract both automated exploitation toolchains and human-operated campaigns.

To minimize identification by scanning platforms, the honeypot avoids artefacts typical of decoy systems. It performs full TCP handshakes, returns realistic status codes, and maintains genuine certificate fingerprints, and avoids honeypot-specific banners or debugging interfaces. Network placement across diverse autonomous systems reduces clustering by reconnaissance tools. Additional mechanisms, such as randomised response timing and natural error handling, further maintain believability. External tests confirm the deployment remains undetected by custom honeypot identification scripts.

4.2. Auxiliary Heterogeneous Dataset

To evaluate generalizability under heterogeneous conditions, we incorporate an independent dataset collected from an internal honeypot platform that emulates diverse web interfaces. The dataset spans 2025-05-01 to 2025-05-30 and contains 3,543,418 HTTP requests generated across 19 cloud deployed sensors.

The platform operates two modules. The attack observation module accepts traffic on all TCP ports, returns

realistic HTTP responses, records incoming requests, and attempts to retrieve malware when commands such as *curl* or *wget* are detected. The response collection module expands device emulation by retrieving HTML content from nearby Internet hosts whenever a previously unseen port–path combination is observed. This enables the honeypot to grow its behavioural repertoire automatically.

4.3. Traffic Characteristics and Labels

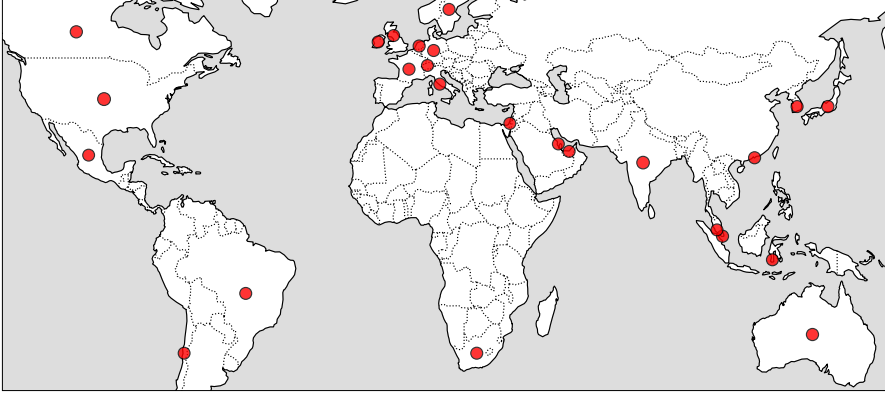
The collected traffic includes reconnaissance scans, exploit attempts, and confirmed malicious activity originating from varied attacker infrastructures. Each session is labelled based on observable actions and execution semantics into one of three categories used throughout our drift analysis:

- **Scan:** Broad reconnaissance such as directory probing, path enumeration, or lightweight sweeps, with no credentials or exploit execution.
- **Attempt:** Targeted access attempts involving credential submission, exploit probing, or requests to sensitive administrative or configuration paths, without confirmed command execution or payload delivery.
- **Intrusion:** Confirmed malicious execution, including command invocation, malware download attempts, or shell-level operations.

Analysis Granularity We process all streams in one hour blocks, which provide sufficient temporal resolution for analysing fine-grained drift dynamics and serve as the core unit for computing block-level drift scores. The granularity is adjustable for real-time deployment. Additional experimental setup and training configuration details appear in Appendix C.

5. Evaluation

This section answers the research questions in Section 2.2. We begin with **RQ1**, evaluating the fidelity of drift indicators in tracking real performance degradation. **RQ2** examines their operational reliability under statistical thresholding. **RQ3** assesses drift-aware adaptation and its impact on long-term detection stability. Finally, **RQ4** evaluates generalisation across heterogeneous traffic sources and attack classes.



Country	Count	Country	Count
US	8,773,005	IN	5,695,172
CA	1,864,316	AU	1,696,078
JP	1,593,329	SG	1,162,571
HK	1,148,428	NL	1,097,017
IT	1,000,939	FR	958,144
DE	855,642	KR	724,774
ZA	670,989	GB	645,807
IE	615,929	ID	589,095
SE	533,374	BR	506,695
IL	495,454	AE	494,552
MY	437,115	BH	410,515
CH	393,284	CL	272,548
MX	241,826		

Figure 4: Coverage of the Primary Honeypot Deployment across 25 host countries (left), and the corresponding request volumes observed per host country in the dataset (right).

5.1. Baselines for Comparison

We compare LANTERN with four representative drift-detection approaches covering the principal methodological families. We compare our framework with four representative state-of-the-art methods selected to cover the major families of drift detection approaches. Table 3 summarizes their main fields, signals, principles, and granularities in a unified view. CADE [8] measures deviation in a contrastive latent space and provides sample-level distance scores. OWAD [5] evaluates shifts in calibrated model outputs through a permutation-based comparison of distributions. Chen et al. [11] introduce a pseudo loss that reflects neighborhood inconsistency within a contrastive representation. Mateen [10] applies a KS-based test on reconstruction error distributions to identify changes in behaviour. These baselines represent contrastive distance methods, calibrated distribution tests, neighborhood uncertainty models, and reconstruction-based detectors.

LANTERN combines predictive entropy and latent typicality to capture confidence erosion and structural deformation within real web traffic. Implementation details appear in Appendix B.

5.2. Indicator Effectiveness Analysis (RQ1)

We first assess how well drift indicators reflect real performance decay. Figure 1 shows the time-aligned evolution of model performance and drift scores over the 7-month evaluation period. All indicators are computed from the same static model trained on the first 192 blocks and applied without adaptation. Accuracy and F1 Score show a gradual decline with intermittent sharp drops, capturing both long-term degradation and transient shocks. These patterns are closely mirrored by the drift indicators, which exhibit consistent upward trends and growing volatility. This alignment confirms that drift scores effectively reflect performance deterioration and changing traffic behaviour, highlighting their value for real-time degradation tracking.

To further validate the effectiveness of each indicator, we measure how well its drift scores align with actual model performance. Table 4 reports the Spearman correlation between each indicator and performance metrics. Our LMT consistently achieves the strongest correlations across all metrics, reaching 0.94 for Accuracy and 0.83

for F1 Score. PE also performs robustly, surpassing all baselines. Among prior methods, CADE and Chen exhibit moderate correlations, while Mateen is weaker. OWAD performs poorly, with near-zero or insignificant correlation across the board. These trends are visually confirmed in Figure 5, where LMT and PE reveal a clear monotonic trend, where drift scores rise as performance drops, while others are noisier or non-informative. This confirms that our indicators more faithfully reflect underlying model degradation.

Takeaway. LMT and PE accurately track real-world degradation, providing high-fidelity drift signals.

5.3. Indicator Reliability Analysis (RQ2)

Having established that our proposed indicators have high fidelity, we next examine whether indicators can operate reliably under statistical thresholding.

5.3.1. Ground Truth Threshold and Evaluation Logic.

To evaluate operational reliability, we first establish a consistent definition of model degradation. We define a groundtruth threshold (T_{GT}) based on overall model performance. For each evaluated method m , we compute $T_m = \text{median}(F1_m) - 1.5 \times \text{std}(F1_m)$, and then take the mean across all methods as the global cutoff: $T_{GT} = \frac{1}{M} \sum_{m=1}^M T_m$. Blocks with $F1 < T_{GT}$ are marked as *drifted*, where $T_{GT} = 0.7053$.

Based on this ground truth, we define how we interpret the detection outcomes. A True Positive (TP) occurs when the detector correctly identifies a degradation period where the model performance drops below T_{GT} . A False Positive (FP) indicates an unnecessary alarm in a stable period, while a False Negative (FN) represents a missed degradation. True Negatives (TN) correspond to correctly identified stable periods.

We detect drift when an indicator deviates from its adaptive baseline by $k \in \{1.5, 2.0, 3.0\}$ times the spread, with the direction determined by performance correlation. Specific statistical formulations (Gaussian vs. MAD) and indicator selection rationale are detailed in Appendix D.

5.3.2. Reliability Results. We selected indicators with Spearman correlation above 0.6 in Table 4, including

TABLE 3: Comparison of drift detection baselines

Method	Main Field	Signal	Principle	Granularity
CADE [8] (USENIX’21)	Android malware	Latent distance	Contrastive latent space	Sample
OWAD [5] (NDSS’23)	Network traffic and system logs	Calibrated distribution, p-value	Permutation test	Distribution
Chen et al. [11] (USENIX’23)	Android malware	Pseudo loss	Contrastive uncertainty	Sample
Mateen [10] (RAID’24)	Network traffic	KS test, p-value	Reconstruction distribution test	Distribution
LANTERN (Ours)	Web stream	PE and LMT	Confidence and latent structure	Block and sample

Notes: Illustration of the granularity: *Sample* means the drift signal is produced for each individual request; *Distribution* means the drift signal is derived from comparing statistical distributions of model outputs or reconstruction errors; *Block* means the drift signal is computed from an aggregated window of requests.

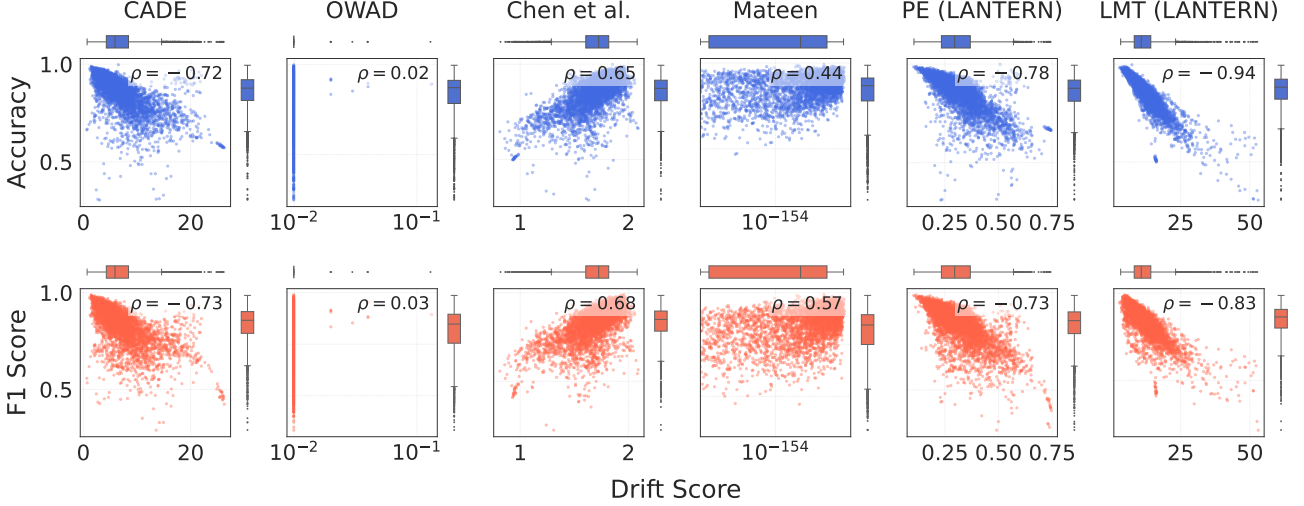


Figure 5: Each subplot visualizes the relationship between drift score (x-axis) and model performance (y-axis; top: accuracy, bottom: F1 score), with marginal boxplots showing the distributions of both variables. OWAD and Mateen use logarithmic x-scales due to their score ranges.

TABLE 4: Indicator–performance correlation.

Method	Correlation with Performance Metrics			
	Accuracy	F1 Score	Precision	Recall
CADE	-0.724	-0.726	-0.710	-0.716
OWAD	0.021	0.032	0.035	0.024
Chen et al.	0.653	0.679	0.679	0.648
Mateen	0.438	0.572	0.571	0.486
PE (LANTERN)	-0.780	-0.733	-0.723	-0.719
LMT (LANTERN)	-0.938	-0.828	-0.787	-0.840

Notes: (1) All p -values are 0.000 except for OWAD, where they are 0.140, 0.023, 0.015, and 0.095 for Accuracy, F1, Precision, and Recall, respectively. (2) The sign of the correlation depends on the definition of each drift score; our focus here is on the **absolute correlation magnitude** to assess indicator sensitivity.

CADE, the Chen formulation, PE, and LMT, and evaluated the detection reliability under the defined groundtruth and prediction logic with $k = 1.5, 2.0$, and 3.0 . The results are summarized in Table 5.

The baseline methods are operationally unviable in a real web environment. CADE retains high correlation but produces an overwhelming number of activations even at 3.0σ , revealing an inherent sensitivity to noise arising from its Euclidean geometry. The Chen formulation performs better yet remains unstable, with 3,581 false positives at 1.5σ and 1,668 at 3.0σ . These activation levels make both baselines impractical for automated operation.

The indicators in our framework present more controlled and informative behaviour. PE reflects belief level confusion and responds when classifier confidence collapses. At the strict 3.0σ level, it reaches a precision

of 1.000 with an $F1_{drift}$ of 0.493, indicating that its alerts concentrate precisely on the most degraded blocks. At 2.0σ , it provides a practical monitoring signal with an $F1_{drift}$ of 0.682, which falls below the groundtruth threshold, and an $F1_{clean}$ of 0.846, confirming stable behaviour during non degraded periods.

LMT captures structural deformation in latent space and delivers a markedly stronger detection profile than the baselines. At the 3.0σ level, it achieves a detection F1 of 0.353, compared with 0.323 for the Chen formulation and 0.163 for CADE. It also obtains the best $F1_{drift}$ at 0.756, showing that its activations align well with true degradation. Its activation rate of 35.55% is more practical compared to CADE’s 94.08% while remaining more selective than the Chen formulation’s 42.03%.

PE therefore provides precise signals for severe collapses but remains inactive elsewhere, and LMT offers more sensitive structural detection but still displays noise. These complementary behaviours show that single channel detection remains insufficient. This motivates a multi-channel monitoring and decision mechanism for robust operation.

5.3.3. Optimal Operational Points. We evaluated the best achievable detection quality for each indicator by scanning its operational threshold range and selecting the threshold that maximizes the detection F1 score. The results appear in Table 6.

Across all indicators, LMT obtains the highest F1 at 0.639, supported by the highest precision at 0.717. This

TABLE 5: Indicator reliability evaluation under different k thresholds.

Method	#Drifts	Rate(%)	Indicator Reliability					Model Performance	
			Precision	Recall	Acc.	F1	TP/FP	F1 _{drift}	F1 _{clean}
CADE (Baseline 1.5σ)	4937	99.44	0.086	0.991	0.090	0.158	424 / 4513	0.835	0.885
CADE (Baseline 2.0σ)	4890	98.49	0.087	0.988	0.099	0.159	423 / 4467	0.834	0.912
CADE (Baseline 3.0σ)	4671	94.08	0.089	0.972	0.141	0.163	416 / 4255	0.831	0.908
Chen (Baseline 1.5σ)	4064	81.85	0.119	0.949	0.274	0.211	483 / 3581	0.813	0.895
Chen (Baseline 2.0σ)	3377	68.02	0.140	0.925	0.407	0.242	471 / 2906	0.799	0.891
Chen (Baseline 3.0σ)	2087	42.03	0.201	0.823	0.646	0.323	419 / 1668	0.766	0.873
PE (LANTERN 1.5σ)	1127	22.70	0.345	0.710	0.820	0.466	391 / 736	0.731	0.863
PE (LANTERN 2.0σ)	406	8.18	0.601	0.443	0.906	0.510	162 / 244	0.682	0.846
PE (LANTERN 3.0σ)	25	0.504	1.000	0.045	0.894	0.087	25 / 0	0.493	0.835
LMT (LANTERN 1.5σ)	3008	60.58	0.138	0.948	0.473	0.241	415 / 2593	0.798	0.906
LMT (LANTERN 2.0σ)	2568	51.72	0.159	0.929	0.559	0.271	407 / 2161	0.785	0.899
LMT (LANTERN 3.0σ)	1765	35.55	0.220	0.888	0.713	0.353	389 / 1376	0.756	0.886

Notes: #Drifts and Rate(%) represent the number and percentage of detected drifted blocks. Indicator reliability metrics measure precision, recall, and accuracy of detection; model performance metrics capture average F1 during drifted and stable periods. Ours show improved balance between detection sensitivity and false alarms.

TABLE 6: Optimal F1 thresholds for each indicator.

Method	Best Threshold	Best F1	Precision	Recall
CADE	9.963	0.413	0.316	0.599
OWAD	0.001	0.000	0.000	0.000
Chen	1.465	0.447	0.440	0.454
Mateen	0.000	0.115	0.070	0.320
PE (LANTERN)	0.444	0.530	0.519	0.541
LMT (LANTERN)	20.433	0.639	0.717	0.576

confirms that structural deviation in latent space aligns closely with true degradation conditions and that LMT maintains stable decision behaviour even under threshold optimisation. PE exhibits complementary behaviour, achieving a maximum F1 of 0.530 with balanced precision and recall, which reflects its ability to surface belief level failures without excessive activation. Both indicators outperform the baselines by a clear margin in terms of overall F1. Given that real-world traffic is inherently dynamic and non-stationary, we retain statistical thresholding as a practical mechanism for adaptive deployment.

Takeaway. PE provides precise severe degradation signals; LMT provides broader structural detection. Both outperform baselines but remain limited individually.

5.4. Stability and Adaptation Analysis (RQ3)

We evaluate whether drift-aware adaptation improves long-term detection stability. This section compares a static baseline (trained once and held fixed) with our dynamic LANTERN configuration, which uses the multi-channel monitoring and decision layers to trigger retraining in response to sustained drift. We analyze the resulting impact on the stability of model performance over time.

5.4.1. Layer Level Behaviour. Building on the reliability analysis in the previous section, where single channel detection showed inherent weaknesses, we adopt a layered multi-channel framework that integrates the complementary strengths of PE and LMT. The reliability results indicate that PE provides stable early signals at 2.0σ without excessive activation, while LMT maintains the

TABLE 7: Detection rates across layers.

Layer	Indicator	Threshold	Detection Rate (%)
Monitor	PE	2.0σ	2.38
Monitor	LMT	3.0σ	23.18
Decision	Fusion Rule	Joint 3.0σ or 8/10 LMT	1.93

most reliable structural detection at 3.0σ . These thresholds therefore form the basis of the monitoring layer, which offers early awareness of belief level and structural drift.

We apply a fusion mechanism for the decision layer, following the adaptive logic in Section 3.3. The layer signals drift only when PE and LMT jointly exceed their 3.0σ levels, or when LMT stays above 3.0σ in eight of the previous ten blocks, indicating continuous structural change. This rule combines strong probabilistic and structural evidence with temporal consistency.

Table 7 summarises the resulting activation rates. PE monitoring remains selective, LMT monitoring provides broader structural visibility, and the decision rule yields a low overall trigger rate of 1.93%. This design preserves early drift awareness while ensuring that adaptation is initiated only under sustained and meaningful degradation.

5.4.2. Model Performance Stability. Figure 6 shows that the static model experiences several collapses throughout the observation period, most visibly in April, June, followed by a persistent decline after September. The adaptive configuration avoids these failures and maintains stable accuracy and F1 across the entire period.

To further quantify this behaviour, Table 8 summarises monthly averages for performance and drift scores. The static configuration shows a continuous decline in accuracy from 0.93 to 0.78 and in F1 from 0.92 to 0.76. In contrast, the adaptive configuration sustains consistently higher performance, exceeding the static setting by more than ten percent in the later period.

We observe gradual degradation in both static and dynamic cases after April, likely caused by rising complexity in the later traffic where more heterogeneous and obfuscated attack patterns emerge. During training, we record a steady rise in the contrastive autoencoder loss, indicating that the latent structure becomes harder to represent as

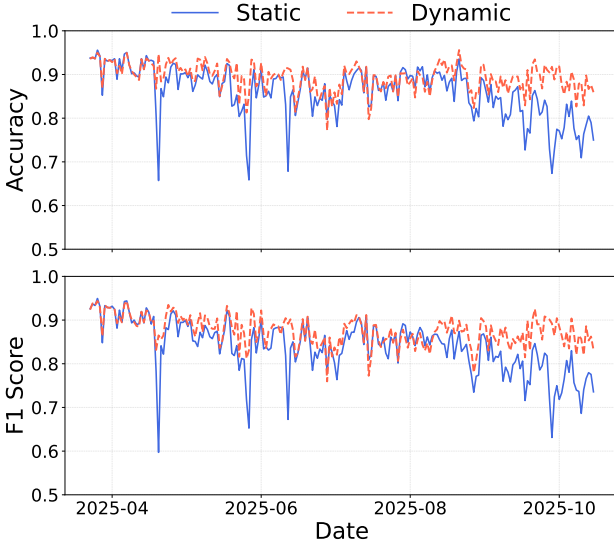


Figure 6: Comparison between static and adaptive models.

TABLE 8: Monthly performance and drift statistics.

Method	Month	Performance		Drift Score		Retrains
		Accuracy	F1	PE	LMT	
Static	2025-03	0.929	0.924	0.188	8.87	0
	2025-04	0.900	0.886	0.239	9.45	0
	2025-05	0.864	0.847	0.258	11.61	0
	2025-06	0.855	0.840	0.305	11.98	0
	2025-07	0.874	0.853	0.314	11.24	0
	2025-08	0.880	0.841	0.310	10.79	0
	2025-09	0.813	0.789	0.390	15.05	0
	2025-10	0.778	0.757	0.389	17.54	0
Dynamic	2025-03	0.930	0.925	0.189	9.64	7
	2025-04	0.917	0.903	0.217	10.84	11
	2025-05	0.899	0.883	0.243	10.28	14
	2025-06	0.879	0.865	0.280	10.02	12
	2025-07	0.883	0.862	0.295	10.16	0
	2025-08	0.896	0.864	0.253	9.81	8
	2025-09	0.889	0.871	0.271	13.12	33
	2025-10	0.874	0.858	0.269	11.46	11

malicious payload diversity grows. Addressing this could involve a larger training window or richer feature capacity, although these directions fall beyond the present scope.

Overall, the adaptive configuration maintains stable performance and prevents severe collapses, confirming effective drift detection and recovery under evolving conditions. This stability is achieved with only 1.93% of blocks triggering model updates, showing that targeted adaptation is both efficient and operationally reliable.

5.4.3. Indicator Drift-Ratio Consistency. Since our adaptation operates at the block level, we also examine the behavioural consistency of the two indicators through their drift ratio distributions in Figure 7. Both indicators demonstrate stable and interpretable patterns, with Entropy showing a slightly lower mean and broader spread (mean = 0.10, std = 0.08) and LMT exhibiting a higher central tendency with tighter variance (mean = 0.14, std = 0.07). These distributions confirm that both indicators remain functional across time and supplement each other by capturing different aspects of drift.

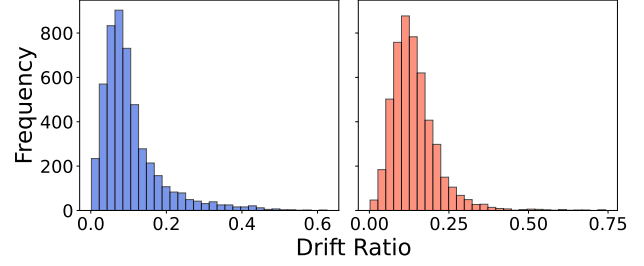


Figure 7: Drift ratio distributions of PE (left) and LMT (right) indicators.

Takeaway. The adaptive framework maintains stable model performance and accurately detects sustained drifts without overreacting to transient changes.

5.5. Generalisation Reliability (RQ4)

Finally, we evaluate whether indicators generalize across heterogeneous sources and multiple attack classes.

5.5.1. Generalisation to Heterogeneous Sources. To evaluate the scalability and generalisation of the framework across diverse data sources, we integrate another honeypot into the live stream during April 2025 and evaluate how static and adaptive models respond.

Figure 8 marks the integration period with a shaded region. During this phase, the static model shows degraded and unstable performance, with F1 averaging 0.829 and accuracy 0.851. In contrast, the adaptive configuration maintains stronger performance, reaching 0.888 F1 and 0.906 accuracy.

We further analyze how the new source influences drift signals. In the static setting, PE increases from 0.222 before and after integration to 0.310 during integration, indicating growing uncertainty in the classifier. LMT behaves differently, decreasing from 5.74 to 4.39, which suggests that the new source introduces subtle but persistent structural anomalies that weaken the latent representation.

Throughout this period, the adaptive system performs 7 updates. Six retrainings are triggered by structural anomalies detected by LMT, and one is triggered by the joint PE and LMT signal. This confirms the structural indicator’s sensitivity in guiding adaptation when facing new, unaligned sources.

5.5.2. Generalisation Across Traffic Classes. To assess whether the structural indicator scales to diverse categories, we analyze LMT drift distributions at the class level. Table 9 summarizes the score distribution across all blocks. The attempt class shows the highest structural volatility, with a wide spread and a 95th percentile above 35, indicating diverse novel behaviours. Intrusion also demonstrates substantial variability, with a high median and a long upper tail that reflects repeated deviations in latent space. In contrast, scan remains the most stable class, with consistently lower LMT scores and a tighter distribution that indicates a narrower and more consistent activity profile.

These patterns show that the indicator captures distinct structural characteristics across classes. The variability

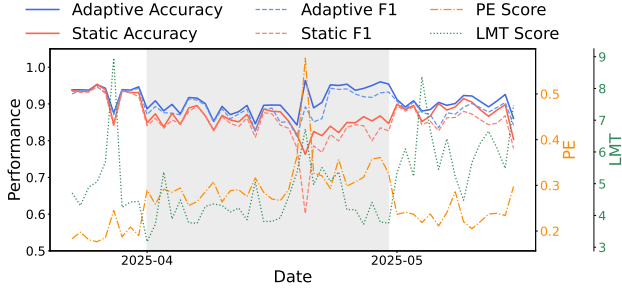


Figure 8: Daily accuracy and F1-Score of static vs. adaptive models under heterogeneous data integration.

across categories demonstrates that the structural signal remains meaningful under different behavioural regimes and generalizes naturally to classes that introduce new or evolving patterns.

TABLE 9: Distribution of LMT scores by traffic class

Class	Mean	Std	Median	95th Petl
Attempt	16.69	13.08	13.68	35.10
Intrusion	13.49	8.67	11.37	28.50
Scan	8.26	4.19	7.25	15.25

Takeaway. LANTERN generalizes across heterogeneous sources and attack classes, maintaining reliable drift signals.

6. Case Study

6.1. Behavioural Analysis of Acute Drift Events

To illustrate block-level behaviour under real traffic, we examine the hourly windows associated with the largest degradations in Figure 6. These short operational intervals reveal how drift accumulates and how indicators respond. The full set and visualisations appear in Appendix E.

Centralised distribution event. Figure 9 shows a clear degradation beginning at 00:00 and lasting until 15:00 happened on 2025-04-20. We inspect this window and find a centralised pattern where 93.7% of all requests originate from a single ASN. Table 10 further shows top 9 user agent strings that appear legitimate, yet each accounts for about 9.5% of total traffic, together exceeding 85%. All requests also follow the same template path structure. Together, these properties reveal a controlled and uniform distribution wave rather than common activity.

Propagation burst event. A separate degraded window appears on 2025-06-12 between 10:00 and 11:00. We observe a short but concentrated propagation burst dominated by architecture-labelled loaders spanning arm, mips, sh, arc, x86 and related variants, none of which appear in the reference buffer. Table 11 lists representative examples. The burst also includes obfuscated staging directories and several configuration or debug probes. These features together indicate an automated multi-architecture propagation phase consistent with embedded system or IoT-focused distribution behaviour.

TABLE 10: Distributed user agent rotation during the drift window on 2025-04-20 00:00-15:00. All types are absent in the reference buffer and appear with uniform frequency.

User Agent Type	Baseline	Drift
iPhone Safari	0.00%	9.58%
Ubuntu Firefox	0.00%	9.56%
Android Chrome	0.00%	9.56%
Windows Edge	0.00%	9.56%
MacOS Safari	0.00%	9.55%
Windows Classic	0.00%	9.54%
iPad Safari	0.00%	9.53%
Windows Firefox	0.00%	9.51%
MacOS Chrome	0.00%	9.51%

Stealth exploitation chain. A third degraded window appears on 2025-10-10 between 01:00 and 14:00. The payloads in this window form a mixed exploitation chain rather than a distribution burst. Table 12 groups them into three parts. The first contains binary style droppers that fetch architecture specific loaders and execute them. The second contains heavily encoded remote execution templates that follow a Log4Shell like pattern combining layered URL encoding, JNDI ldap redirection and a Base64 segment expanding to a shell based cryptomining installer. The third contains probes for misconfiguration and device management panels, including environment files, debug profilers and router interfaces. This combination reflects an access test plus encoded RCE plus post exploitation workflow. Similar mixtures of multi-architecture loaders and encoded command chains appear in recent reporting on the RedTail cryptomining campaign [17].

Taken together, the three windows show distinct behaviours behind degradation. The centralised event reflects coordinated distribution, the propagation burst shows multi-architecture loader activity, and the final window exposes an encoded exploitation chain. These cases show that our indicators surface operational behaviour directly, offering insights beyond numerical drift signals.

6.2. Longitudinal Drift-Exploit Analysis

To investigate which adversarial activities coincide with drift events, we perform a longitudinal analysis of all drifted blocks by matching their HTTP request payloads against the exploit-signature corpus collected by our internally hosted honeypot over the same period. This allows us to identify which exploit surfaces appear within drifted traffic and may contribute to the structural changes captured by our system. Figure 10 highlights two categories of exploit surfaces observed over time: *persistent* and *emerging*. The signatures shown represent only the intersection between drifted traffic and our honeypot’s signature set; the full space of attacks present in drifted blocks is likely broader.

Persistent surfaces. A small number of exploit vectors persist throughout the entire observation window. The dominant one is the well-known PHPUnit remote code execution path (CVE-2017-9841), which targets the `eval-stdin.php` endpoint widely deployed across PHP and CMS ecosystems. A second long-running signature corresponds to a suspected Yi IOT XY-3820

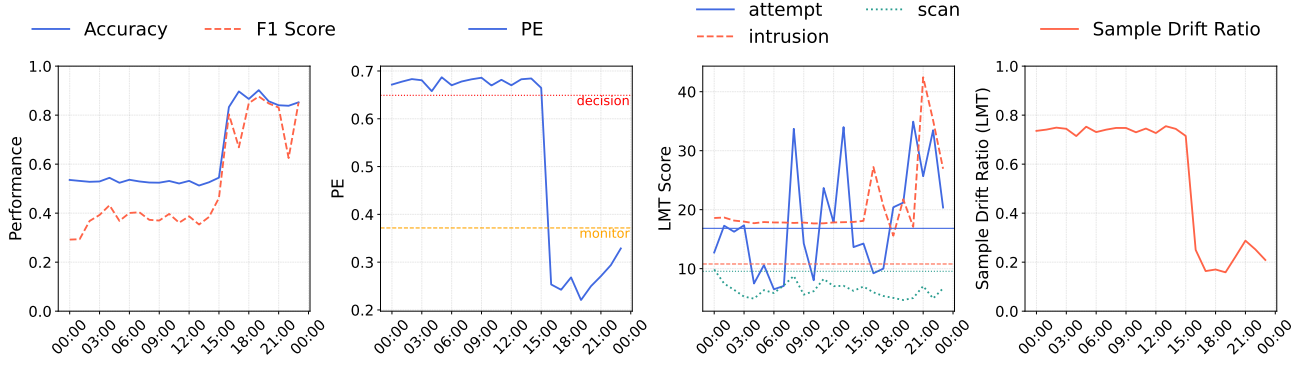


Figure 9: 24-Hour Drift Evolution on 2025-04-20. From left to right: (1) Model performance over time (Accuracy and F1 Score);(2) PE score, with monitor and decision thresholds; (3) LMT scores per class (attempt, scan, intrusion) with class-wise thresholds; (4) LMT sample drift ratio, indicating the proportion of unseen samples per block.

TABLE 11: Representative URI signatures in the 10:00 to 11:00 propagation burst on 2025-06-12. These payloads were unseen during training and reveal a short but intense multi-vector distribution wave.

Payload Category	Examples
Multi architecture loaders	/resgod.ppc, /resgod.x86, /boatnet.arm, /boatnet.sh4, /bins/mips, /bins/arm7, /neon.m68k, /boatnet.x86_64
Obfuscated staging paths	/dwrioej/neon.mips, /dwrioej/neon.arc, /dwrioej/neon.x86_64, /hiddenbin/space.i686, /co, /dss, /586
Misconfiguration and debug probes	/.env, /.env.save, /owa/auth/x.js, /?xdebug_session_start, /lang/custom/sbin/init

TABLE 12: Representative payloads during the attempt spike on 2025-10-10 from 01:00 to 14:00. Payloads are grouped by functional role and all appear exclusively in the drift window.

Payload Category	Examples
Binary style dropper execution	/device.rsp?opt=sys&cmd=wget...boatnet.arm7, /resgod.ppc, /boatnet.sh4, /resgod.mips
Log4Shell-style RCE (Encoded)	/t%28%27...jndi:ldap...base64...%27%29, /t%2528...jndi:ldap...base64...%2529
Misconfiguration and weakness probes	/.env, /cms/.env, /app_dev.php/_profiler/phpinfo, /cgi-bin/luci/stok=/locale, /geoserver/web/

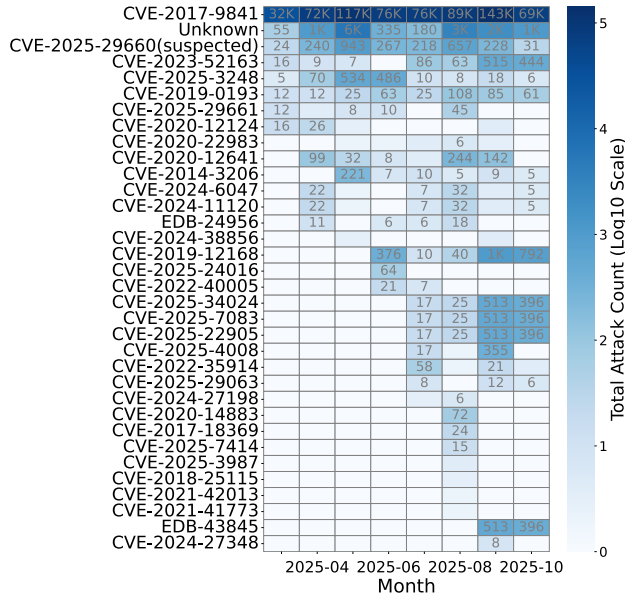


Figure 10: Monthly evolution of drift-associated exploit signatures (2025-03-15 to 2025-10-15).

vulnerability (likely CVE-2025-29660), which first appears in March, peaks at 943 matched events in May, and continues to recur in subsequent months.

Emerging surfaces. Other exploit signatures appear later and grow in prevalence over time. CVE-2025-3248

emerges in April and surges in May and June, largely driven by Langflow code-execution attempts. CVE-2019-12168, associated with command injection on Four Faith F3x24 routers via `/bapply.cgi`, first appears in June and increases sharply again in September. A final cluster arrives in July, including CVE-2025-7083, CVE-2025-22905, and CVE-2025-34024, all exploiting Belkin and Edimax extender endpoints such as `/goform/mp`; these jointly spike in September and October. Because our measurement interval ends on 2025-10-15, the upward trend in October may understate their full trajectory.

This analysis reveals which attacker behaviours are temporally aligned with drift events. Figure 10 provides a temporal overview of these relationships, while Appendix F details the corresponding signature patterns.

7. Related Work

Drift Paradigms in Security. Concept drift describes changes in data distributions or feature label relations over time and is a primary cause of model degradation [2], [21]. Security studies often follow the feature shift and label shift view when modelling temporal evolution in malware or intrusion datasets [3], [7], [22], [23], while others emphasise *distribution drift*, where the input distribution moves without altering class semantics [4], [5], [10]. These formulations capture statistical movement but overlook the behavioural evolution that drives real degradation. In web intrusion settings, attackers continually modify

payload syntax, obfuscation routines, URI templates and staging patterns [19], [20], reshaping latent structure even when surface patterns seem stable. Work on normality shift, such as OWAD [5], models changes in benign activity but yields population summaries that are sensitive to small fluctuations and provide limited semantic insight. These limitations indicate that security relevant drift often stems from behavioural processes outside standard statistical taxonomies, motivating the *behavioural perspective* taken in this study.

Web Intrusion Detection. Web intrusion detection often begins with manually engineered rules or signature based filters, which offer precision but suffer from low recall [24]. Learning based systems are therefore widely adopted, including recurrent and convolution models [25], Markov mixture formulations [26], lexical feature refinement [27] and adversarial robust analysers [28]. Further efforts target specific behaviours through URL based detection, malicious redirection analysis, static heuristic classifiers, guided exploration of malicious pages, detection of web shells and malicious plugins and few shot models for cross site scripting analysis [29]–[33]. These systems improve attack coverage but remain static and offer no insight into long term drift, and many still rely on sample level cues or handcrafted components that generalise poorly under evolving traffic.

Web Exploitation Studies. A complementary line of research analyses web exploitation behaviour at scale, revealing how attacker tooling evolves across platforms and ecosystems. Large studies of web shells and malicious plugins document how compromised interfaces and extension ecosystems support persistent footholds [34], [35]. Measurements of online exploitation further characterise shifts in tactics, infrastructure and payload delivery across campaigns, including automated evaluation and exploitation workflows in web applications [36], [37]. Long term honeynet analyses extend this view, showing multi year behavioural evolution as attackers adopt exploratory actions, modify botnet techniques, prefer newly registered autonomous systems and probe for honeypot presence [38]. Additional efforts analyse zero day behaviour in the wild, extract indicators of compromise and study weaknesses in server side request defences [39]–[41]. These studies illustrate the dynamic and structured nature of web exploitation but focus on behavioural characterisation rather than how such evolution affects learning based intrusion detection or contributes to long term drift.

Representation Learning for Security. Representation learning [42] has become central to modern security analytics, with deep encoders, temporal embeddings and graph representations providing richer structure than traditional features. Word embeddings reveal dynamic attack phases across stages [43], [44], while GNN-based provenance models [45] and snapshot embeddings combined with contextual sequence models [46] support advanced threat detection. Adaptive latent models further improve analysis under non-stationary traffic [16], [47]. Latent diagnosis methods such as CADE detect drift through single point Euclidean deviation [8], yet single point distance is insufficient to capture structural deformation of latent geometry under adversarial evolution. This limitation motivates indicators that measure geometric typicality and decision stability during drift.

8. Discussion

Label Efficiency. This study uses fully labelled data to verify model behaviour, but such annotations are rarely available in operational intrusion detection, where large traffic volumes and rapidly evolving malicious activity make manual labelling impractical. Drift indicators provide an alternative form of weak supervision by capturing behavioural shifts without relying on explicit labels. These signals support stable model performance in unlabeled environments and offer actionable points for human inspection, allowing operators to review periods of change and understand their underlying causes.

Model Adaptation. Our study adopts an eight day reference window as the baseline, selected through empirical evaluation showing that this period captures sufficient behavioural diversity to support stable detection. Prior work similarly observes that large volumes of traffic are not required to establish an effective anomaly baseline [48]. Adaptation occurs when drift indicators confirm sustained behavioural change, allowing the model to refresh its parameters and preserve stability. More advanced approaches such as meta learning for fast adaptation [49], [50], transfer learning [51], ensemble learning [52] or incremental learning under continuous streams [53] may provide finer control over model evolution, but these techniques remain outside the scope of this study.

Honeypot Scope and Extensibility Our evaluation uses two independent honeypot deployments that both provide HTTP based traffic, while the primary infrastructure supports multiple protocols including SSH, FTP and Telnet. We focus on HTTP in this study, but the broader deployment context shows that the framework is not constrained to a single protocol. Additional data sources such as real web services, enterprise gateways and traffic from controlled IoT laboratories can integrate directly into the pipeline, a capability we have already validated through heterogeneous inter and intra experiments in Section 5.5. The pipeline remains modular across domains and requires only adjustments in feature extraction while preserving its drift diagnosis and adaptation components.

9. Conclusion

We present LANTERN, a dual-channel drift monitor that delivers reliable and interpretable degradation signals for web intrusion detection. By combining Predictive Entropy with Latent Mahalanobis Typicality, LANTERN captures both decision-level confusion and structural deformation; sources of failure that existing detectors routinely miss. Across seven months of real HTTP(S) traffic, state-of-the-art methods proved operationally unusable, often producing thousands of false positives, while LMT achieved a striking $r = 0.938$ correlation with F1. Integrated into an adaptive pipeline, LANTERN prevents the severe collapses seen in static models and preserves over 10% higher F1 in the later period. Case studies show that it exposes the behavioural roots of drift, from coordinated distribution waves to multi-architecture propagation and encoded exploitation chains. These results position LANTERN as a practical, production-ready foundation for resilient intrusion detection under evolving attacker behaviour.

References

- [1] SentinelOne. (2025) Cyber security statistics. [Online]. Available: <https://www.sentinelone.com/cybersecurity-101/cybersecurity/cyber-security-statistics/>
- [2] F. Bayram, B. S. Ahmed, and A. Kassler, "From concept drift to model degradation: An overview on performance-aware drift detectors," *Knowledge-Based Systems*, vol. 245, p. 108632, 2022.
- [3] G. Andresini, F. Pendlebury, F. Pierazzi, C. Loglisci, A. Appice, and L. Cavallaro, "Insomnia: Towards concept-drift robustness in network intrusion detection," in *Proceedings of the 14th ACM workshop on artificial intelligence and security*, 2021, pp. 111–122.
- [4] M. Dragoi, E. Burceanu, E. Haller, A. Manolache, and F. Brad, "Anoshift: A distribution shift benchmark for unsupervised anomaly detection," *Advances in Neural Information Processing Systems*, vol. 35, pp. 32 854–32 867, 2022.
- [5] D. Han, Z. Wang, W. Chen, K. Wang, R. Yu, S. Wang, H. Zhang, Z. Wang, M. Jin, J. Yang *et al.*, "Anomaly detection in the open world: Normality shift detection, explanation, and adaptation," in *NDSS*, 2023.
- [6] J. Zhang, J. Xu, B. Can, and Y. Fan, "React: Residual-adaptive contextual tuning for fast model adaptation in threat detection," in *Proceedings of the ACM on Web Conference 2025*, 2025, pp. 2488–2499.
- [7] R. Jordaney, K. Sharad, S. K. Dash, Z. Wang, D. Papini, I. Nouretdinov, and L. Cavallaro, "Transcend: Detecting concept drift in malware classification models," in *26th USENIX security symposium (USENIX security 17)*, 2017, pp. 625–642.
- [8] L. Yang, W. Guo, Q. Hao, A. Ciptadi, A. Ahmadzadeh, X. Xing, and G. Wang, "{CADE}: Detecting and explaining concept drift samples for security applications," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2327–2344.
- [9] F. Maggi, W. Robertson, C. Kruegel, and G. Vigna, "Protecting a moving target: Addressing web application concept drift," in *International workshop on recent advances in intrusion detection*. Springer, 2009, pp. 21–40.
- [10] F. Alotaibi and S. Maffei, "Mateen: Adaptive ensemble learning for network anomaly detection," in *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses*, 2024, pp. 215–234.
- [11] Y. Chen, Z. Ding, and D. Wagner, "Continuous learning for android malware detection," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 1127–1144.
- [12] D. Arp, M. Spreitzenbarth, M. Hubner, H. Gascon, K. Rieck, and C. Siemens, "Drebin: Effective and explainable detection of android malware in your pocket," in *Ndss*, vol. 14, no. 1, 2014, pp. 23–26.
- [13] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani *et al.*, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," *ICISSp*, vol. 1, no. 2018, pp. 108–116, 2018.
- [14] A. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *37th annual IEEE/IFIP international conference on dependable systems and networks (DSN'07)*. IEEE, 2007, pp. 575–584.
- [15] J. Song, H. Takakura, Y. Okabe, M. Eto, D. Inoue, and K. Nakao, "Statistical analysis of honeypot data and building of kyoto 2006+ dataset for nids evaluation," in *Proceedings of the first workshop on building analysis datasets and gathering experience returns for security*, 2011, pp. 29–36.
- [16] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: an ensemble of autoencoders for online network intrusion detection," *arXiv preprint arXiv:1802.09089*, 2018.
- [17] M. Donde. (2025, September) Aide uncovers redtail: Persistent cryptomining with apt tactics. [Online]. Available: <https://globalcyberalliance.org/aide-data-redtail/>
- [18] S. Chen, R. Wang, X. Wang, and K. Zhang, "Side-channel leaks in web applications: A reality today, a challenge tomorrow," in *2010 IEEE Symposium on Security and Privacy*. IEEE, 2010, pp. 191–206.
- [19] K. P. Dyer, S. E. Coull, and T. Shrimpton, "Marionette: A programmable network traffic obfuscation system," in *24th USENIX Security Symposium (USENIX Security 15)*, 2015, pp. 367–382.
- [20] J. E. Díaz-Verdejo, R. E. Alonso, A. E. Alonso, and G. Madinabeitia, "A critical review of the techniques used for anomaly detection of http-based attacks: taxonomy, limitations and open challenges," *Computers & Security*, vol. 124, p. 102997, 2023.
- [21] A. Kantchelian, S. Afroz, L. Huang, A. C. Islam, B. Miller, M. C. Tschantz, R. Greenstadt, A. D. Joseph, and J. D. Tygar, "Approaches to adversarial drift," in *Proceedings of the 2013 ACM workshop on Artificial intelligence and security*, 2013, pp. 99–110.
- [22] D. W. Fernando and N. Komninos, "Fesad ransomware detection framework with machine learning using adaption to concept drift," *Computers & Security*, vol. 137, p. 103629, 2024.
- [23] S. Yang, X. Zheng, J. Li, J. Xu, X. Wang, and E. C. Ngai, "Recda: Concept drift adaptation with representation enhancement for network intrusion detection," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 3818–3828.
- [24] S. T. Jan, Q. Hao, T. Hu, J. Pu, S. Oswal, G. Wang, and B. Viswanath, "Throwing darts in the dark? detecting bots with limited data using neural data augmentation," in *2020 IEEE symposium on security and privacy (SP)*. IEEE, 2020, pp. 1190–1206.
- [25] R. Tang, Z. Yang, Z. Li, W. Meng, H. Wang, Q. Li, Y. Sun, D. Pei, T. Wei, Y. Xu *et al.*, "Zerowall: Detecting zero-day web attacks through encoder-decoder recurrent neural networks," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 2479–2488.
- [26] Y. Song, A. D. Keromytis, and S. Stolfo, "Spectrogram: A mixture-of-markov-chains model for anomaly detection in web traffic," 2009.
- [27] M. Mimura, "Adjusting lexical features of actual proxy logs for intrusion detection," *Journal of Information Security and Applications*, vol. 50, p. 102408, 2020.
- [28] J. M. Vidal, M. A. S. Monge, and S. M. M. Monterrubio, "Espada: Enhanced payload analyzer for malware detection robust against adversarial threats," *Future Generation Computer Systems*, vol. 104, pp. 159–173, 2020.
- [29] H. Mekky, R. Torres, Z.-L. Zhang, S. Saha, and A. Nucci, "Detecting malicious http redirections using trees of user browsing activity," in *IEEE INFOCOM 2014-IEEE Conference on Computer Communications*. IEEE, 2014, pp. 1159–1167.
- [30] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond black-lists: learning to detect malicious web sites from suspicious urls," in *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2009, pp. 1245–1254.
- [31] C. Seifert, I. Welch, and P. Komisarczuk, "Identification of malicious web pages with static heuristics," in *2008 Australasian Telecommunication Networks and Applications Conference*. IEEE, 2008, pp. 91–96.
- [32] L. Invernizzi, P. M. Comparetti, S. Benvenuti, C. Kruegel, M. Cova, and G. Vigna, "Evilseed: A guided approach to finding malicious web pages," in *2012 IEEE symposium on Security and Privacy*. IEEE, 2012, pp. 428–442.
- [33] H. Pan, Y. Fang, W. Guo, Y. Xu, and C. Wang, "Few-shot graph classification on cross-site scripting attacks detection," *Computers & Security*, vol. 140, p. 103749, 2024.
- [34] O. Starov, J. Dahse, S. S. Ahmad, T. Holz, and N. Nikiforakis, "No honor among thieves: A large-scale analysis of malicious web shells," in *Proceedings of the 25th International Conference on World Wide Web*, 2016, pp. 1021–1032.
- [35] R. P. Kasturi, J. Fuller, Y. Sun, O. Chabklo, A. Rodriguez, J. Park, and B. Saltaformaggio, "Mistrust plugins you must: A {Large-Scale} study of malicious plugins in {WordPress} marketplaces," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 161–178.
- [36] D. Canali and D. Balzarotti, "Behind the scenes of online attacks: an analysis of exploitation behaviors on the web," in *20th Annual Network & Distributed System Security Symposium (NDSS 2013)*, 2013, pp. n-a.

- [37] A. Stasinopoulos, C. Ntantogian, and C. Xenakis, “Commix: automating evaluation and exploitation of command injection vulnerabilities in web applications,” *International Journal of Information Security*, vol. 18, no. 1, pp. 49–72, 2019.
- [38] C. Munteanu, Y. B. Suriyanarayanan, G. Smaragdakis, A. Feldmann, and T. Fiebig, “Attacks come to those who wait: Long-term observations in an ssh honeynet,” in *ACM on Internet Measurement Conference*. ACM, 2025.
- [39] L. Bilge and T. Dumitraş, “Before we knew it: an empirical study of zero-day attacks in the real world,” in *Proceedings of the 2012 ACM conference on Computer and communications security*, 2012, pp. 833–844.
- [40] C. Sabottke, O. Suciu, and T. Dumitraş, “Vulnerability disclosure in the age of social media: Exploiting twitter for predicting {Real-World} exploits,” in *24th USENIX security symposium (USENIX security 15)*, 2015, pp. 1041–1056.
- [41] M. Wessels, S. Koch, G. Pellegrino, and M. Johns, “{SSRF} vs. developers: A study of {SSRF-Defenses} in {PHP} applications,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 6777–6794.
- [42] P. Stojanov, Z. Li, M. Gong, R. Cai, J. Carbonell, and K. Zhang, “Domain adaptation with invariant representation learning: What transformations to learn?” *Advances in Neural Information Processing Systems*, vol. 34, pp. 24 791–24 803, 2021.
- [43] Y. Shen and G. Stringhini, “{ATTACK2VEC}: Leveraging temporal word embeddings to understand the evolution of cyberattacks,” in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 905–921.
- [44] A. Alsaheel, Y. Nan, S. Ma, L. Yu, G. Walkup, Z. B. Celik, X. Zhang, and D. Xu, “{ATLAS}: A sequence-based learning approach for attack investigation,” in *30th USENIX security symposium (USENIX security 21)*, 2021, pp. 3005–3022.
- [45] M. U. Rehman, H. Ahmadi, and W. U. Hassan, “Flash: A comprehensive approach to intrusion detection via provenance graph representation learning,” in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 3552–3570.
- [46] F. Yang, J. Xu, C. Xiong, Z. Li, and K. Zhang, “{PROGRAPHER}: An anomaly detection system based on provenance graph embedding,” in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 4355–4372.
- [47] X. Wang, “Enidrift: A fast and adaptive ensemble system for network intrusion detection under real-world drift,” in *Proceedings of the 38th annual computer security applications conference*, 2022, pp. 785–798.
- [48] R. Estepa, J. E. Diaz-Verdejo, A. Estepa, and G. Madinabeitia, “How much training data is enough? a case study for http anomaly-based intrusion detection,” *IEEE Access*, vol. 8, pp. 44 410–44 425, 2020.
- [49] C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” in *International conference on machine learning*. PMLR, 2017, pp. 1126–1135.
- [50] A. Kumagai, T. Iwata, H. Takahashi, and Y. Fujiwara, “Meta-learning for robust anomaly detection,” in *International Conference on Artificial Intelligence and Statistics*. PMLR, 2023, pp. 675–691.
- [51] B. Neyshabur, H. Sedghi, and C. Zhang, “What is being transferred in transfer learning?” *Advances in neural information processing systems*, vol. 33, pp. 512–523, 2020.
- [52] L. L. Minku and X. Yao, “Ddd: A new ensemble approach for dealing with concept drift,” *IEEE transactions on knowledge and data engineering*, vol. 24, no. 4, pp. 619–633, 2011.
- [53] J. He, R. Mao, Z. Shao, and F. Zhu, “Incremental learning in online scenario,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 13 926–13 935.

Appendix A.

Data Availability

The primary honeypot data used in this study is processed and analyzed in situ at the data collector through

controlled interfaces provided by a major non profit honeypot operator with whom we maintain an active research collaboration. No raw data is exported outside the operator’s premises at any stage of the study.

Sensitive information about the deployment is anonymized to avoid collateral risk to participating networks. We do not disclose hosting locations, infrastructure identifiers or configuration details, as such information could enable fingerprinting or targeted misuse of the honeypots. Only high level descriptions of protocol coverage and traffic characteristics are provided, consistent with responsible disclosure practices in measurement based security research.

Appendix B.

SOTA Implementations

This section summarises the external drifting indicators used as baselines in our study. For clarity, we denote latent vectors by z_i , class labels by y_i , class subsets by $S_c = \{z_i : y_i = c\}$, reconstruction errors by x_i , classifier probabilities by p_{ic} , and Euclidean latent distances by d_{ij} . A block contains T samples indexed by t , and all block-level drift scores are computed as averages over these T samples. Only the core formulations are reported.

B.1. CADE

CADE measures deviation from class-aligned latent geometry. For each class c , the centroid and deviation statistics are μ_c and MAD_c .

Each sample obtains the minimum normalized deviation

$$A_t = \min_c \frac{\|z_t - \mu_c\|_2 - \tilde{d}_c}{\text{MAD}_c}.$$

The block drift score is

$$\text{CADE}_{\text{score}} = \frac{1}{T} \sum_{t=1}^T A_t.$$

B.2. OWAD

OWAD detects drift by calibrating reconstruction errors into a probability space and comparing the calibrated distributions of the reference and test blocks.

Given reconstruction errors x_i , isotonic regression produces a monotone mapping f , and calibrated scores are

$$p_i = 1 - f(x_i).$$

Let O_k and E_k denote the observed and expected histogram counts over K bins. The likelihood-ratio G statistic is

$$G = \sum_k (O_k + \delta) \left[\log(O_k + \delta) - \log(E_k + \delta) \right].$$

A permutation test evaluates significance, and the drift score is the resulting p-value:

$$\text{OWAD}_{\text{score}} = p_{\text{perm}}.$$

B.3. Chen Pseudo-loss

The Chen pseudo-loss was originally introduced as a training objective to enhance representation robustness; in our setting it is repurposed as a drift indicator that captures instability in classifier confidence and latent neighbourhood structure. Given classifier probabilities p_{ic} , the confidence pseudo-loss is

$$L_{ce}(i) = -\log(\max_c p_{ic}).$$

The confidence term in Chen focuses solely on the largest probability and therefore measures the sharpness of the top decision. Our predictive entropy instead aggregates the full distribution and reflects broader decision level ambiguity. As a result, Chen’s pseudo loss acts as a local confidence signal, whereas our entropy indicator captures global uncertainty across all classes.

Latent distances $d_{ij} = \|z_i - z_j^{\text{ref}}\|_2$ define neighbourhood sets $\hat{P}(i)$ and $\hat{N}(i)$. The neighbourhood inconsistency term is

$$L_{hc}(i) = \frac{1}{|\hat{P}(i)|} \sum_{j \in \hat{P}(i)} \max(0, d_{ij} - m) + \frac{1}{|\hat{N}(i)|} \sum_{j \in \hat{N}(i)} \max(0, 2m - d_{ij}).$$

The combined pseudo-loss is

$$S(i) = L_{hc}(i) + \lambda_{ce} L_{ce}(i),$$

and the block drift score is

$$\text{Chen}_{\text{score}} = \frac{1}{T} \sum_{i=1}^T S(i).$$

B.4. Mateen

Mateen detects drift by comparing reconstruction error distributions from the reference and test blocks using the Kolmogorov–Smirnov statistic. Given empirical CDFs F_{ref} and F_{test} ,

$$D_{\text{KS}} = \sup_x |F_{\text{ref}}(x) - F_{\text{test}}(x)|, \quad p_{\text{KS}} = \text{KS-pval}.$$

The Mateen drift score is simply the KS p-value:

$$\text{Mateen}_{\text{score}} = p_{\text{KS}}.$$

Appendix C. Experimental Setup

This section summarises the configuration used for feature extraction, representation learning and classifier training in our study. The setup follows the same pipeline across all experiments, with hyperparameters fixed unless otherwise stated.

Feature Engineering. HTTP records are transformed into numerical vectors using a TF–IDF encoder. Selected fields including protocol, method, status, URI, user-agent, referer and body are concatenated into a single text string before vectorisation. A vocabulary of 2000 terms is learned from the reference window, and all blocks are encoded using the same vectoriser.

Contrastive Autoencoder. Inputs are encoded using a contrastive autoencoder with fully connected layers and ReLU activations. The latent dimension is fixed at 128. Training minimises the sum of a mean squared reconstruction loss and a contrastive loss that attracts same class samples and repels different class samples under a margin of 10. The model is trained with the Adam optimiser, a learning rate of 1e-3, a batch size of 512 and an early stopping patience of 2 epochs.

Latent Classifier. Latent vectors extracted from the reference window are used to train a random forest classifier with 100 estimators and parallel processing enabled. This classifier operates entirely in the latent space and produces the predictions used for performance evaluation.

Evaluation Protocol. Traffic is segmented into non overlapping 1 hour blocks. Each block yields latent vectors, reconstruction errors and classifier predictions, from which drift scores and performance metrics are computed. Heterogeneous evaluations follow the same configuration with additional sensors introduced within a region or across regions.

Appendix D. Statistical Threshold Selection

This section summarises the statistical criteria used for adaptive thresholding across all drifting indicators. All thresholds follow a robust deviation formulation based on either median and MAD or mean and standard deviation, depending on the indicator.

D.1. PE Threshold

PE is computed from classifier outputs on the reference window, where the model is already trained on all classes and therefore produces a tightly concentrated distribution with low variance. This distribution is light tailed and well captured by the mean and standard deviation. For this reason, PE is the only indicator that uses a Gaussian-style baseline. The monitoring and decision thresholds are:

$$T_{\text{PE}} = \mu_{\text{ref}} + k\sigma_{\text{ref}},$$

with $k = 2.0$ for monitoring and $k = 3.0$ for decision.

D.2. Thresholds for Other Indicators

All other indicators (LMT, CADE and Chen et al.) exhibit heavy-tailed reference behaviour due to class geometry and intra-class variability in the latent space. These distributions are not well approximated by mean and standard deviation and instead require robust statistics.

We therefore use a unified threshold rule for all remaining indicators:

$$T_{\text{others}} = \text{median}_{\text{ref}} + k \cdot \text{MAD}_{\text{ref}},$$

where $k = 3.0$ for monitoring and decision.

D.3. Ground Truth Performance Threshold

For reference in performance aligned analysis, performance degradation is defined by

$$T_{\text{perf}} = \text{median}(F1_{\text{ref}}) - 1.5 \cdot \text{std}(F1_{\text{ref}}),$$

TABLE 13: Confusion-matrix definition for drift evaluation.

Case	Condition
True Positive (TP)	Detected drift, $F1 < T_{GT}$
False Positive (FP)	Detected drift, $F1 \geq T_{GT}$
False Negative (FN)	No drift, $F1 < T_{GT}$
True Negative (TN)	No drift, $F1 \geq T_{GT}$

where $F1_{ref}$ denotes block level F1 scores in the reference window.

This threshold follows a statistical process control style rule, where $\text{median}(F1_{ref})$ provides a robust centre and $1.5 \cdot \sigma$ serves as a moderate deviation limit. Blocks falling beyond this control bound represent performance levels unlikely to arise from normal operational variability.

Based on this ground truth, the confusion matrix in Table 13 defines how detection outcomes are interpreted.

Appendix E. Detailed Case Study

To provide finer detail for the case study, we present visualisations for the five most significant drift days identified in our evaluation. Each figure summarises the block level evolution of performance, predictive entropy, latent structure and the proportion of drifted samples. Table 14 provides a consolidated overview of the five events, and we describe each day in turn below.

E.1. A Coordinated Attack Behind the Drift on 2025-04-20

Figure 9 shows a long drift period from 00:00 to 15:00. During this window the daily F1 score drops to 0.516 and accuracy to 0.640. LMT remains far above its threshold throughout the 15-hour interval, and the drifted sample ratio stays above 0.7. PE also rises above 0.65 and remains unstable for the entire window.

Inspection of the raw traffic confirms a coordinated intrusion campaign. A single ASN accounts for 93.7% of all requests, compared with only 0.3% in the baseline. The traffic rotates across nine entirely new user agent strings, each contributing roughly 9 to 10 percent of requests. All dominant URIs follow a single unseen pattern of the form `wp-login.php%20%7c%20[hostname]`, with the pipe character acting as a probe marker.

These characteristics generate a dense and novel latent cluster, explaining the sustained elevation of LMT, and the conflicting mix of legitimate looking user agents with malicious probes drives the prolonged PE spike. Overall, the day reflects a clear joint structural and belief drift driven by a high volume, structurally uniform attack.

E.2. The Erratic Attempt on 2025-09-28

Figure 11 shows a highly unstable pattern where the daily F1 score drops to 0.624. The curve does not collapse for a long period but instead exhibits several sharp crashes. These crashes align with repeated, large spikes in the LMT attempt score, often far above the decision threshold, while

other classes remain relatively steady. The drifted sample ratio also fluctuates sharply, confirming intermittent structural failures.

Inspection of the underlying traffic shows that no single source dominates this day. The top 20 ASNs list a diverse mix of new and many baseline zero networks, producing a fragmented infrastructure. User agent activity is also mixed, with a new Firefox 47 bot at 10.9%, libredtail http at 5.1% and the Palo Alto Networks scanner at 1.1%. The URI distribution reveals multiple concurrent campaigns, including a ten percent OpenWrt Luci scan, probes such as `/qr-code.php` and `/sputnicv.php`, and a direct boatnet malware download sequence targeting ARM based devices.

This combination produces a volatile structural signal: one block is dominated by Luci scanning, the next by malware delivery. The rapid change of dominant attempt behaviour explains the repeated LMT surges and the unstable performance is clear in Figure 11.

E.3. Interleaved reconnaissance scan and a short Mirai-like botnet intrusion burst on 2025-05-27

Figure 12 shows a mixed drift pattern where different classes dominate at different times. From 00:00 to 15:00 the LMT scan score stays continuously above its threshold, and performance shows a mild but steady decline. Scan traffic increases from 20.3% in the baseline to 44.1% during this window. The top ASN rises from 0.2% to 27.5%, and several new ASNs appear with contributions of 2.6% and 2.3%, indicating a distributed reconnaissance operation. User agent activity is dominated by the missing nan field, which grows from 6.7% to 27.1%, together with malformed Python and curl based identifiers. The leading URIs include `wp-login.php`, `xmlrpc.php`, `/1.txt` and `/debug/pprof`, consistent with broad scanning and fingerprinting across common web services.

Between 07:00 and 09:00 a separate pattern emerges where LMT for intrusion rises sharply. During this short interval, the top three ASNs contribute 22.1%, 21.8% and 19.0% respectively, forming a coordinated multi cloud distribution of payloads. The dominant user agent is a Windows Chrome 95.0 string that increases from 0.39% to 18.4%, and several tool driven identifiers such as `axios/0.27.2` and `Keydrop.io/1.0` appear. In the meantime, we identified short, obfuscated paths (e.g., `/mqppc`, `/rjfe686`, `/rrrdsl`) or raw ELF drop-pers (e.g. `/bot.spc`, `/skid.arm6`) consistent with loader behaviour (Table 15). Combined, these payloads account for more than 20% of total traffic during the spike, indicating a high-volume dropper delivery campaign.

Overall, 2025-05-27 reflects an interleaved drift event: a long reconnaissance phase that produces sustained structural deviation, followed by a short intrusion burst that triggers both indicators through rapid payload changes and coordinated source activity.

E.4. Obfuscated Intrusion with Browser Covered Payloads on 2025-10-10

Figure 13 shows a sustained drift window from 01:00 to 14:00. During this period the intrusion based LMT

TABLE 14: Summary of major drift events and diagnoses. We highlight the affected classes, drift patterns, attack trends, root causes, and insights from our framework.

Date	Performance	Drift Type	Dominant Class	Trend Summary	Root Cause	Framework Insight
2025-04-20	F1 0.516, Acc 0.640	Structured, Sustained, Novel	Intrusion, Attempt	Distributed attack from 00:00–15:00	Centralized botnet in single ASN (93.7%), 9 spoofed UAs, templated URI probes	LMT + PE co-triggered
2025-09-28	F1 0.624, Acc 0.668	Volatile, Intermittent	Attempt	Erratic spikes throughout day	Fragmented ASNs, diverse toolchains and payloads, direct botnet malware download attempt.	LMT spiked, PE elevated
2025-05-27	F1 0.640, Acc 0.634	Mixed-phase with broad scan and intrusion burst	Scan, Intrusion	Broad scan 00:00–15:00, Mirai-like burst 07:00–09:00	Cloud-based recon, missing UAs, malware droppers with arch tags	LMT separates multi-prong intrusion pattern
2025-10-10	F1 0.672, Acc 0.701	Sustained, Coordinated	Intrusion	Long active intrusion and slight attempt during 01:00-14:00	Obfuscated intrusion with browser masked payloads and cloaked scanning activity	Coordinated alignment with LMT and PE
2025-06-06	F1 0.678, Acc 0.686	Stealthy, Temporary spike	Attempt	Attempt spike at 10:00–11:00 amid active scan	Obfuscated IoT malware campaign using browser mimicking UAs and multi-arch binaries	LMT captured structural anomaly

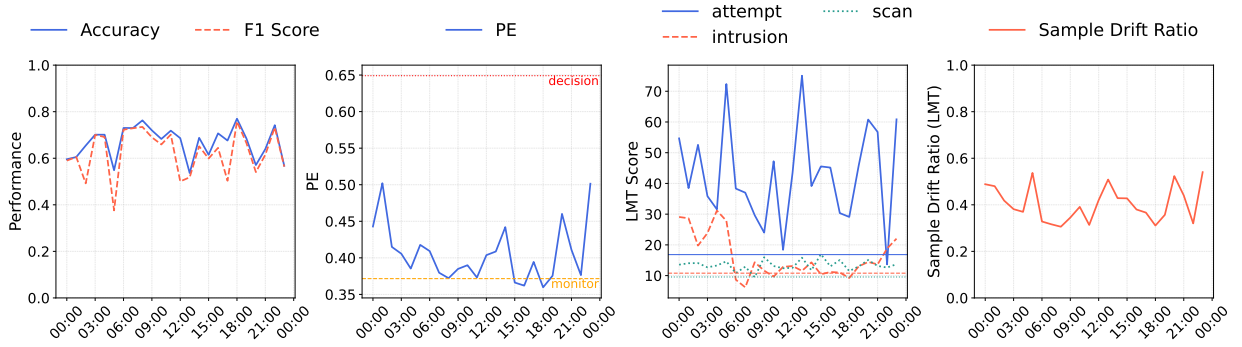


Figure 11: 24-Hour Drift Evolution on 2025-09-28. Extremely erratic attempt score with massive spikes well above its threshold. Other classes remain relatively stable but generally above the threshold.

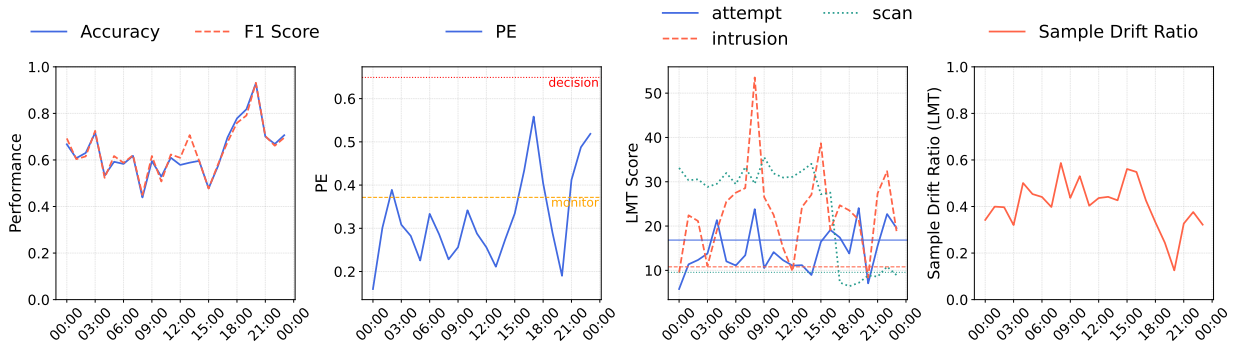


Figure 12: 24-Hour Drift Evolution on 2025-05-27. This mixed-phase drift shows two distinct behaviours: a broad scan phase from 00:00 to 15:00 with sustained over-threshold LMT scan scores, followed by a short but sharp intrusion spike between 07:00 and 09:00.

score remains well above its threshold, PE stays elevated and performance drops repeatedly. Scan scores are over threshold and the traffic in this period increases from

20.3% to 39.0%, but the dominant drivers of the drift are intrusion class and attempt class behaviour.

Infrastructure changes show broad expansion. The

TABLE 15: New URI signatures in the 07:00–09:00 intrusion spike on 2025-05-27. These architecture-labeled malware droppers were all unseen during training and collectively formed over 20% of observed traffic.

Payload Pattern	Examples
Architecture-specific binaries	/bins/sora.x86_64, /bins/pmips, /bins/spc, /bins/parm6, /bins/sh4, /bins/sora.i686, /bins/ppc
Obfuscated short paths	/mqar6, /mqsh, /vejfa5, /rrrdsl, /mqar5, /mqms, /mqppc, /mqar4
Direct ELF drops	/bot.spc, /efea6, /skid.arm6, /mqm68k, /bins/pm68k, /bins/pmpsl, /bejv86

leading provider in the window rises from 0.05% to 28.6%, several others increase by roughly 5 to 7%, and nearly two hundred new providers appear, indicating a wide multi source distribution. User agent changes are dominated by browser mimicry. A Windows Chrome variant climbs from near zero to 18.9%, nan reaches 12.5%, and multiple scanning tools appear, including libredtail-http at 4.7%, Go-http-client/1.1 at 3.6% and ivre-masscan at 1.0%. The URI distribution shifts sharply, with /blog/wp-login.php increasing from 0 to 18.9%, / rising from 12.9% to 25.6%, and multiple new probes such as /admin/config.php, /.git/config, /qr-code.php, /sputnicv.php and /hudson emerging.

Overall, 2025-10-10 reflects a single, high volume and structurally consistent intrusion campaign. The combination of large scale novelty, browser impersonation and mixed probing behaviour produces the sustained LMT elevation, while the conflicting tool and browser signatures explain the prolonged rise in PE.

Payload interpretation. The representative payloads in Table 12 illustrate three complementary behaviours that co-occur in the 01:00–14:00 window. While we cannot assert that they originate from a single toolchain, their functional alignment suggests the presence of a broader exploitation workflow. The first group consists of binary-style droppers that retrieve architecture-specific loaders; the second group contains deeply encoded remote-execution templates resembling Log4Shell-style triggers; and the third group probes configuration and management endpoints. These behaviours together match the mixed exploitation pattern reflected in the sustained LMT and PE elevation in Figure 13.

Zoom-in: Encoded Log4Shell-style cryptomining routine. A closer inspection of the encoded RCE payloads reveals a small but highly coherent component inside the drift window. We observe 238 such requests from three clients, with one accounting for 94.5% (225 samples). Across these sources there are 1,016 related records, dominated by attempt traffic (75.1%) and scan (24.6%), indicating a probing-heavy context around a fixed exploit module. The payloads follow the same Log4Shell-style pattern: layered URL encoding, an embedded jndi:ldap redirection, and a long Base64 segment. Three nearly identical variants point to the same LDAP endpoint and each appears 75 times, forming a fully repeated and internally homogeneous exploit template.

Decoding the Base64 component yields a shell command that downloads and executes a remote cryptominer installer, consistent with XMRig-style deployment. This behaviour aligns with a Log4Shell-triggered cryptomining routine rather than general-purpose injection. Two of the three clients also issue probe traffic, but none produce a

probe → RCE → dropper sequence, suggesting that this module operates as a standalone exploit within the broader 01:00–14:00 drift.

E.5. Obfuscated Botnet Propagation on 2025-06-12

Figure 14 shows a drift event where scan activity remains elevated for most of the day, while a sharp and isolated surge in the attempt class appears between 10:00 and 11:00. The LMT score rises far above its threshold during this hour and performance drops accordingly. Ground truth analysis confirms that this window contains strong structural novelty. Scan traffic increases from 20.3% to 44.7%, the attempt proportion remains high at 24.8%, and more than 8 new AS organisations appear. Several sources rise by 2% to 12% and the top contributor climbs from 0.26% to 21.0%. User agent behaviour is heavily obfuscated, with nan rising from 6.8% to 21.4% and multiple browser styled or tool based strings expanding by between 3% and 8%. URI changes expose a concentrated propagation phase: more than 20 architecture specific binaries appear, including ARM, MIPS, SH4 and x86 variants, all of them baseline zero. Additional new paths such as .env, login pages and router related entries indicate misconfiguration probing around the same period.

The combined evidence points to a short but intense botnet propagation burst embedded inside an already unstable traffic day. LMT isolates the hourly spike with clarity and captures the multi-architecture malware distribution pattern that defines this event, showing how structural signals preserve sensitivity even when belief signals are suppressed by obfuscation.

Appendix F. Representative Samples of Exploit Surfaces

CVE-2017-9841: PHPUnit eval path execution Requests consistently target the eval-stdin.php execution path across many directory layouts including /vendor/phpunit/..., /lib/phpunit/..., and /admin/phpunit/.... The request body embeds a PHP system call that downloads and executes a remote script. These repeated directory variants reflect widespread deployments of PHPUnit components in CMS and PHP based services.

CVE-2025-29660 (suspected): Yi IOT XY-3820 based execution endpoint Requests use /execute and upload a command string that downloads and runs an external payload. The structure and behaviour align with patterns observed in Yi IOT XY 3820 based devices. The signature persists across many months with stable formatting and content.

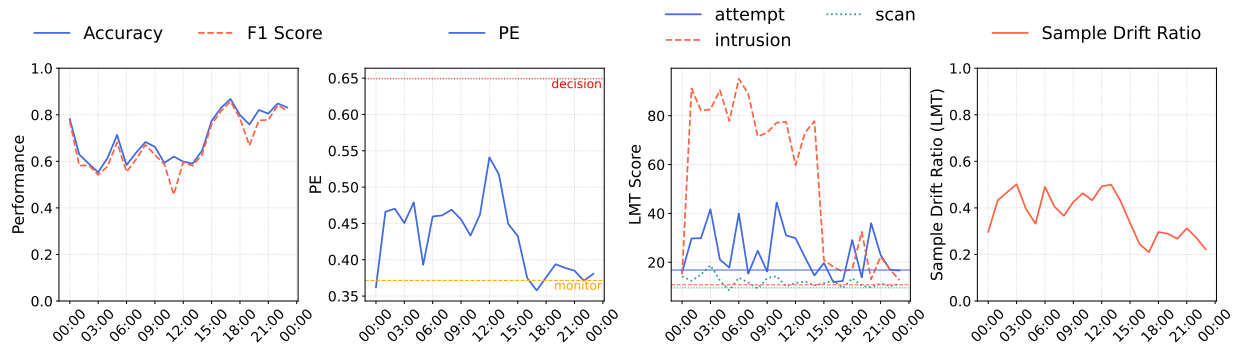


Figure 13: 24-Hour Drift Evolution on 2025-10-10. From 01:00 to 14:00, we observe continuous, elevated intrusion and attempt activity, accompanied by performance degradation, PE exceeding the monitor threshold, and a rise in the drifted sample ratio, signaling a coordinated and sustained behavioural shift.

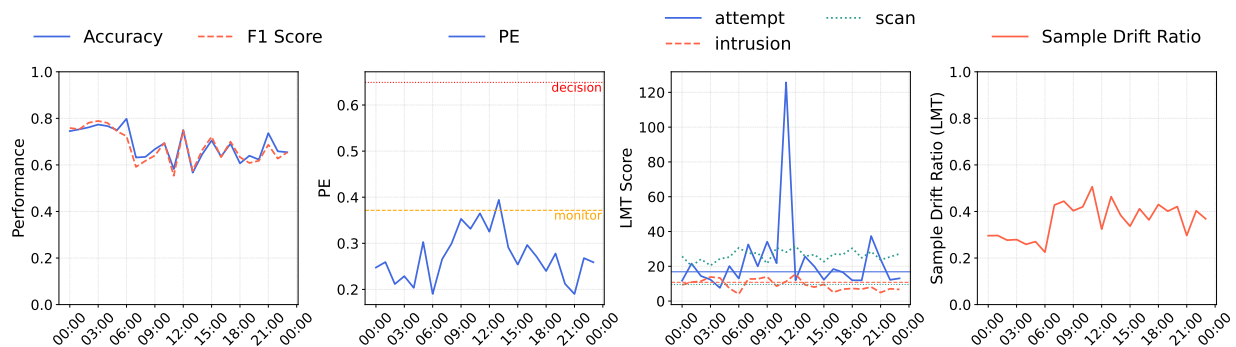


Figure 14: 24-Hour Drift Evolution on 2025-06-12. Scan scores remain consistently above threshold, with a sharp attempt surge at 10:00–11:00 and a post 06:00 rise in drift ratio that coincides with performance decline.

CVE-2025-3248: Langflow execution workflow Requests target `/api/v1/validate/code` and pass a Python expression crafted to spawn a subprocess that fetches a remote script. This behaviour aligns with weaknesses in Langflow based application environments. Activity increases sharply between May and June.

CVE-2019-12168: Four Faith F3x24 diagnostics interface. Requests use `/bapply.cgi` with command injection through the `ping_ip` parameter, repurposing the diagnostics page to download and execute remote code. The pattern appears from June onward with a strong rise in September.

CVE-2025-7083, CVE-2025-22905, CVE-2025-34024: Belkin and Edimax extender families Requests target `/goform/mp` and inject commands via the `command` parameter to fetch and launch an external payload. These signatures emerge in July and spike in September and October.